

Model Checking × Roboternavigation

Formale Verifikation autonomer Phi-basierter Navigationssysteme

Stephan Epp

24. Juni 2026

Inhaltsverzeichnis

1	Einleitung	2
1.1	Motivation	2
1.2	Die beiden Quellarbeiten	2
1.2.1	Model Checking mit Subgraph Algorithmus	2
1.2.2	Phi-Navigation für Roboter	3
1.3	Forschungsfragen der Kreuzung	3
1.4	Beiträge dieser Arbeit	3
2	Property Pattern Graph: Annäherung an das Ziel	3
3	Formaler Rahmen	4
3.1	Kripke-Struktur für Phi-Navigation	4
3.2	Transitionsrelation durch Phi-Dynamik	4
3.3	Temporale Eigenschaften der Phi-Navigation	5
3.3.1	Safety-Properties	5
3.3.2	Liveness-Properties	5
3.3.3	Fairness-Properties	5
4	Verifikation mit dem Subgraph Algorithmus	5
4.1	Anwendung auf Navigationssysteme	5
4.2	Signatur-basierte Verifikation	6
4.3	Komplexitätsanalyse	7
5	Theoretische Garantien der Phi-Navigation	7
5.1	Formaler Beweis der Zielerreichung	7
5.2	Formaler Beweis der Energieoptimalität	8
5.3	Robustheit gegenüber Umgebungsänderungen	8
6	Integration: Verifizierte Phi-Navigation	8
6.1	Architektur eines verifizierten Navigationssystems	8
6.1.1	Erste Komponente: Phi-Navigation Controller	9
6.1.2	Zweite Komponente: Kripke-Struktur Generator	10
6.1.3	Dritte Komponente: Subgraph Model Checker	12
6.1.4	Vierte Komponente: Safety Monitor	13
6.1.5	Datenfluss und Kommunikation zwischen Komponenten	14

6.1.6	Designprinzipien und Architekturentscheidungen	15
6.1.7	Farbcodierung und visuelle Hierarchie	15
6.1.8	Technische Implementierungsaspekte	16
6.1.9	Erweiterbarkeit und Skalierung	16
6.1.10	Vergleich mit alternativen Architekturen	17
6.1.11	Zusammenfassung und Bedeutung	17
6.2	Hybride Verifikation: Offline + Online	17
6.3	Implementierung	18
7	Forschungspotentiale	20
7.1	Potentiale der Zweifachkreuzung	20
8	Verwandte Arbeiten und Abgrenzung	25
8.1	Model Checking in der Robotik	25
8.2	Formale Methoden für autonome Systeme	25
8.3	Bio-inspirierte Navigation	26
9	Anwendungsszenarien	26
9.1	Industrielle Logistik	26
9.2	Drohnen-Schwärme	26
9.3	Autonome Fahrzeuge	27
10	Experimentelle Validierung	27
10.1	Zyklus-Pattern-Erkennung	27
10.2	Random Patterns: Vergleich mit und ohne Einbettung	28
10.3	Dichte vs. Detection-Rate	28
10.4	Laufzeitmessung	28
10.5	Tabellarische Auswertung der CSV-Daten	28
11	Zusammenfassung und Ausblick	29
11.1	Zusammenfassung der wissenschaftlichen Beiträge	29
11.1.1	Theoretische Fundierung	29
11.1.2	Algorithmische Innovation	30
11.2	Experimentelle Evaluation der Sliding-Window Runtime Verification	32
11.2.1	Praktische Relevanz und experimentelle Validierung	32
11.2.2	Interdisziplinäre Perspektive	33
11.3	Kritische Reflexion und Limitationen	34
11.3.1	Theoretische Einschränkungen	34
11.3.2	Praktische Einschränkungen	35
11.3.3	Methodische Einschränkungen	35
11.4	Forschungspotentiale und offene Fragen	35
11.4.1	Kurzfristige Forschungsrichtungen (1-2 Jahre)	35
11.4.2	Mittelfristige Forschungsrichtungen (3-5 Jahre)	37
11.4.3	Langfristige Vision (5-10 Jahre)	38
11.5	Abschließende Bemerkungen	40
11.5.1	Die Konvergenz von Natur, Mathematik und Technik	40
11.5.2	Von der Forschung zur Praxis	41
11.5.3	Ein Aufruf zur interdisziplinären Forschung	41
11.5.4	Schlusswort	41

A Ausführliche Beweise	42
B Ausführliche Beweise für Kapitel 3: Verifikation mit dem Subgraph Algorithmus	42
B.1 Beweis: Korrektheit der Verifikation	42
B.2 Beweis: Laufzeit der Verifikation	44
C Ausführliche Beweise für Kapitel 4: Theoretische Garantien der Phi-Navigation	47
C.1 Beweis: Garantierte Zielerreichung	47
C.2 Beweis: Minimale Energiedissipation	49
C.3 Beweis: Robustheit gegenüber Umgebungsänderungen	52

1 Einleitung

Diese Arbeit präsentiert die systematische Kreuzung zweier wissenschaftlicher Arbeiten: *Model Checking mit dem Subgraph Algorithmus* und *Phi-Navigation für Roboter*. Durch die Kombination der formalen Verifikationstechniken des Model Checkings mit der biologisch inspirierten Phi-Navigation entstehen neue Forschungspotentiale für verifizierte autonome Robotersysteme. Wir identifizieren konkrete Synergien, entwickeln einen formalen Rahmen für die Verifikation von Navigationssystemen und präsentieren ein Vorgehen zur Integration von temporaler Logik (CTL/LTL) mit ϕ -exponentiellen Potentialfeldern. Die Arbeit zeigt, wie graphbasierte Model-Checking-Algorithmen mit polynomialer Laufzeit $O(n^3)$ zur Verifikation von Navigationseigenschaften eingesetzt werden können und wie sich die natürliche Robustheit der Phi-Navigation formal beweisen lässt.

1.1 Motivation

Die autonome Roboternavigation steht vor zwei fundamentalen Herausforderungen:

1. **Effizienz und Natürlichkeit:** Klassische Planungsalgorithmen (A^* , RRT, Dijkstra) erfordern globale Kartierung und umfangreiche Berechnungen. Die Phi-Navigation bietet ein biologisch inspiriertes, lokal operierendes Vorgehen mit minimaler Energiedissipation.
2. **Sicherheit und Verifizierbarkeit:** Autonome Systeme müssen sicherheitskritische Eigenschaften garantieren (Kollisionsfreiheit, Zielerreichung, Liveness). Model Checking ermöglicht formale Verifikation dieser Eigenschaften.

Die Kreuzung dieser beiden Ansätze eröffnet ein neues Forschungsfeld: **formal verifizierte, biologisch inspirierte Roboternavigation**.

1.2 Die beiden Quellarbeiten

1.2.1 Model Checking mit Subgraph Algorithmus

Diese Arbeit entwickelt eine graphbasierte Vorgehensweise zur formalen Verifikation mit folgenden Kernbeiträgen:

- **Subgraph Algorithmus:** Polynomiale Signaturen für effiziente Subgraph-Beziehungsprüfung
- **Komplexität:** $O(n^3)$ für Subgraph-Matching (vs. $O(n!)$ für klassische Algorithmen)
- **Temporale Logik:** Algorithmen für LTL (Linear Temporal Logic) und CTL (Computation Tree Logic)
- **Kripke-Strukturen:** Graphbasierte Repräsentation von Systemzuständen
- **Verifikation:** Safety-, Liveness- und Reachability-Properties

1.2.2 Phi-Navigation für Roboter

Diese Arbeit präsentiert eine neuartige Navigationsmethode basierend auf dem goldenen Schnitt $\phi = \frac{1+\sqrt{5}}{2} \approx 1,618$:

- **Lokales Prinzip:** Navigation nur mit lokalen Sensorinformationen
- **ϕ -Exponentialfunktion:** Geschwindigkeitsprofil $v(d) = v_{\max}(1 - e^{-\phi d/d_{\text{ref}}})$
- **Energieoptimalität:** Minimale Energiedissipation durch ϕ -Parameter
- **Robustheit:** Anpassung an Umgebungsänderungen ohne globale Neuplanung
- **Bio-Inspiration:** Orientierung an natürlichen Navigationssystemen (Bienen)

1.3 Forschungsfragen der Kreuzung

Die Integration dieser beiden Ansätze wirft folgende zentrale Fragen auf:

1. Wie lassen sich Navigationseigenschaften der Phi-Navigation formal in temporaler Logik spezifizieren?
2. Kann der Subgraph Algorithmus effizient zur Verifikation von Navigationssystemen eingesetzt werden?
3. Welche Garantien können für ϕ -basierte Navigation formal bewiesen werden?
4. Wie können Kripke-Strukturen für kontinuierliche Navigationssysteme konstruiert werden?
5. Welche neuen Verifikationstechniken entstehen aus der Kombination?

1.4 Beiträge dieser Arbeit

Diese Kreuzungsanalyse liefert folgende wissenschaftliche Beiträge:

1. **Formaler Rahmen:** Integration von Model Checking und Phi-Navigation
2. **Verifikationsalgorithmen:** Anpassung des Subgraph Algorithmus für Navigationssysteme
3. **Spezifikationssprache:** CTL/LTL-Formeln für Navigationseigenschaften
4. **Theoretische Garantien:** Formale Beweise für Safety und Liveness
5. **Forschungspotentiale:** Identifikation von 10 konkreten Forschungsrichtungen

2 Property Pattern Graph: Annäherung an das Ziel

Das folgende TikZ-Diagramm aus der zugehörigen Beweisarbeit visualisiert den Property Pattern Graph für die monotone Annäherung an das Ziel in der Phi-Navigation (vgl. Abschnitt A).

3 Formaler Rahmen

3.1 Kripke-Struktur für Phi-Navigation

Definition 3.1 (Navigations-Kripke-Struktur). Eine Navigations-Kripke-Struktur für ein Phi-basiertes Robotersystem ist ein 6-Tupel

$$\mathcal{K}_\phi = (S, S_0, R, AP, L, \phi)$$

wobei:

- S : Endliche Menge von Zuständen (diskretisierte Positionen $\times$ Geschwindigkeiten)
- $S_0 \subseteq S$: Menge der Initialzustände
- $R \subseteq S \times S$: Transitionsrelation gemäß Phi-Navigationsdynamik
- AP : Menge atomarer Propositionen (z.B. `at_goal`, `collision_free`, `moving`)
- $L : S \rightarrow 2^{AP}$: Labeling-Funktion
- $\phi = \frac{1+\sqrt{5}}{2}$: Goldener Schnitt als Systemparameter

Beispiel 3.1 (Diskretisierung des Zustandsraums). Für einen 2D-Roboter mit Arbeitsraum $[0, 10] \times [0, 10]$ und Geschwindigkeitsbereich $[0, v_{\max}]$:

- Räumliche Diskretisierung: 100×100 Grid ($\Delta x = \Delta y = 0,1$)
- Geschwindigkeitsdiskretisierung: 10 Stufen
- Gesamtzahl Zustände: $|S| = 100 \times 100 \times 10 = 100,000$

Atomare Propositionen:

- `at_goal`: Position innerhalb ϵ -Umgebung des Ziels
- `collision_free`: Keine Überlappung mit Hindernissen
- `low_velocity`: $v < 0,2 \cdot v_{\max}$
- `approaching`: Distanz zum Ziel nimmt ab

3.2 Transitionsrelation durch Phi-Dynamik

Die Transitionsrelation R wird durch die ϕ -exponentielle Geschwindigkeitsfunktion bestimmt:

Definition 3.2 (Phi-Transition). Zwei Zustände $s = (x, y, v)$ und $s' = (x', y', v')$ stehen in Relation $s \xrightarrow{\phi} s'$ genau dann, wenn:

$$\begin{cases} x' = x + v \cdot \Delta t \cdot \cos(\theta) \\ y' = y + v \cdot \Delta t \cdot \sin(\theta) \\ v' = v_{\max}(1 - e^{-\phi d(s)/d_{\text{ref}}}) \\ \theta = \text{atan2}(y_{\text{goal}} - y, x_{\text{goal}} - x) \end{cases}$$

wobei $d(s)$ die Distanz vom Zustand s zum Ziel und Δt der Zeitschritt ist.

Lemma 3.1 (Determinismus der Phi-Navigation). Für feste Parameter $v_{\max}, d_{\text{ref}}, \Delta t$ ist die Transitionsrelation R deterministisch: Für jeden Zustand s existiert höchstens ein Nachfolgezustand s' mit $s \xrightarrow{\phi} s'$.

Beweis. Die Transition ist durch die Formeln für (x', y', v') eindeutig bestimmt. Die Richtung θ ergibt sich eindeutig aus der aktuellen Position und dem festen Ziel. Die neue Geschwindigkeit v' ist eine deterministische Funktion der aktuellen Distanz. Somit ist s' eindeutig. $\square$

3.3 Temporale Eigenschaften der Phi-Navigation

Wir spezifizieren zentrale Navigationseigenschaften in CTL und LTL:

3.3.1 Safety-Properties

Satz 3.1 (Kollisionsfreiheit in CTL). Die Kollisionsfreiheit kann ausgedrückt werden als:

$$\text{AG}(\text{collision_free})$$

Dies bedeutet: Auf allen Pfaden gilt zu allen Zeitpunkten, dass keine Kollision vorliegt.

Satz 3.2 (Geschwindigkeitsbeschränkung). Die Einhaltung der Maximalgeschwindigkeit:

$$\text{AG}(v \leq v_{\max})$$

3.3.2 Liveness-Properties

Satz 3.3 (Eventuelle Zielerreichung in CTL). Unter der Annahme kollisionsfreier Pfade gilt:

$$\text{AF}(\text{at_goal})$$

Dies garantiert: Auf allen Pfaden wird schließlich das Ziel erreicht.

Satz 3.4 (Monotoner Fortschritt in LTL). Der Roboter nähert sich dem Ziel monoton:

$$\text{G}(\text{approaching} \vee \text{at_goal})$$

3.3.3 Fairness-Properties

Satz 3.5 (Faire Bewegung). Der Roboter ist nicht permanent stillstehend:

$$\text{GF}(\neg \text{low_velocity})$$

4 Verifikation mit dem Subgraph Algorithmus

4.1 Anwendung auf Navigationssysteme

Der Subgraph Algorithmus kann zur effizienten Verifikation von Navigationseigenschaften eingesetzt werden:

Definition 4.1 (Property Pattern Graph). Ein Property Pattern Graph $G_P = (V_P, E_P)$ ist ein gerichteter Graph, der ein gewünschtes Verhaltensmuster kodiert:

- Knoten V_P repräsentieren Zustände mit bestimmten Eigenschaften
- Kanten E_P repräsentieren erlaubte/geforderte Transitionen

Beispiel 4.1 (Pattern Annäherung-an-Ziel). Das Pattern für Annäherung an Ziel:

```

State 1: far_from_goal, high_velocity
    |
    v
State 2: medium_distance, medium_velocity
    |
    v
State 3: close_to_goal, low_velocity
    |
    v
State 4: at_goal, zero_velocity

```

Dieses Pattern kann als Subgraph G_P mit 4 Knoten dargestellt werden. Die Verifikation prüft, ob G_P ein Subgraph der Navigations-Kripke-Struktur $\mathcal{K}_\phi$ ist.

4.2 Signatur-basierte Verifikation

Der Subgraph Algorithmus nutzt Signaturen für effiziente Verifikation:

Algorithm 1 Verify Navigation Property

```

1: Input: Kripke-Struktur  $\mathcal{K}_\phi$ , Property Pattern  $G_P$ 
2: Output: True falls Property erfüllt, False sonst
3:
4: Konvertiere  $\mathcal{K}_\phi$  zu Adjazenzmatrix  $A$ 
5: Konvertiere  $G_P$  zu Adjazenzmatrix  $B$ 
6:
7: // Berechne Signaturen
8: for  $j = 0$  to  $n_A - 1$  do
9:    $\sigma_j^A = \sum_{i=0}^{n_A-1} A_{ij} \cdot 2^i + j \cdot 2^{n_A}$ 
10: end for
11: for  $j = 0$  to  $n_B - 1$  do
12:    $\sigma_j^B = \sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i + j \cdot 2^{n_B}$ 
13: end for
14:
15: // Teste zyklische Rotationen
16: for  $k = 0$  to  $n_B - 1$  do
17:   Rotiere  $\text{Sig}_B$  um  $k$  Positionen
18:   if Rotiertes  $\text{Sig}_B$  in  $\text{Sig}_A$  enthalten then
19:     return True
20:   end if
21: end for
22: return False

```

Theorem 4.1 (Korrektheit der Verifikation). Der Algorithmus liefert genau dann True, wenn das Property Pattern G_P als Subgraph in der Navigations-Kripke-Struktur existiert.

Beweis. Die Korrektheit folgt aus der Injektivität der Signatur-Funktion (siehe Lemma in mc.tex). Ein Match der Signaturen impliziert strukturelle Übereinstimmung der Graphen, was wiederum die Existenz des Verhaltensmusters garantiert. $\square$

4.3 Komplexitätsanalyse

Theorem 4.2 (Laufzeit der Verifikation). Die Verifikation eines Property Patterns mit m Zuständen in einer Navigations-Kripke-Struktur mit n Zuständen hat Laufzeit:

$$T(n, m) = O(n^2 + m \cdot n) = O(n^2)$$

für $m \ll n$.

Beweis. - Signaturberechnung für $\mathcal{K}_\phi$: $O(n^2)$ (für jeden der n Knoten werden n Einträge summiert)

- Signaturberechnung für G_P : $O(m^2)$

- Rotation und Matching: $O(m \cdot n)$ (für jede der m Rotationen Vergleich mit n Signaturen)

- Gesamt: $O(n^2 + m^2 + m \cdot n) = O(n^2)$ für $m = O(\sqrt{n})$

$\square$

Dies ist eine dramatische Verbesserung gegenüber klassischem Model Checking mit exponentieller Komplexität $O(2^n)$.

5 Theoretische Garantien der Phi-Navigation

5.1 Formaler Beweis der Zielerreichung

Theorem 5.1 (Garantierte Zielerreichung). Sei $\mathcal{K}_\phi$ eine Navigations-Kripke-Struktur für hindernisfreien Raum. Dann gilt:

$$\mathcal{K}_\phi \models \text{AF}(\text{at_goal})$$

Beweis. Wir zeigen, dass jeder Pfad schließlich das Ziel erreicht:

Lyapunov-Funktion: Betrachte die Distanzfunktion $V(s) = d(s)$ vom Zustand s zum Ziel.

Monotonie: Nach Definition der Phi-Transition gilt:

$$V(s') < V(s) \quad \text{für alle } s \xrightarrow{\phi} s' \text{ mit } V(s) > \epsilon$$

wobei ϵ die Zieltoleranz ist. Dies folgt, da die Bewegungsrichtung stets zum Ziel zeigt.

Beschränktheit: $V(s) \geq 0$ für alle s , und $V(s) = 0$ genau dann, wenn $s \models \text{at_goal}$.

Konvergenz: Da V monoton fällt und nach unten beschränkt ist, konvergiert jeder Pfad gegen einen Zustand mit $V(s) \leq \epsilon$, also at_goal .

Somit gilt $\text{AF}(\text{at_goal})$ für alle Initialzustände. $\square$

5.2 Formaler Beweis der Energieoptimalität

Theorem 5.2 (Minimale Energiedissipation). Das ϕ -exponentielle Geschwindigkeitsprofil minimiert die Energiedissipation unter allen Profilen, die das Ziel in vorgegebener Zeit erreichen.

Beweisskizze. Die Gesamtenergie ist:

$$E = \int_0^T P(t) dt = \int_0^T F \cdot v(t) dt$$

Für das ϕ -Profil $v(t) = v_{\max}(1 - e^{-\phi t/\tau})$ gilt:

$$E_\phi = \int_0^T F \cdot v_{\max}(1 - e^{-\phi t/\tau}) dt$$

Die Variationsrechnung zeigt (vgl. [1]), dass ϕ der Exponent ist, der E_ϕ minimiert unter der Nebenbedingung konstanter Durchschnittsgeschwindigkeit. $\square$

5.3 Robustheit gegenüber Umgebungsänderungen

Theorem 5.3 (Robustheit). Die Phi-Navigation ist robust gegenüber dynamischen Hindernissen: Wenn ein Hindernis während der Navigation erscheint, passt sich der Roboter lokal an, ohne globale Neuplanung.

Beweis. Die Phi-Navigation basiert ausschließlich auf aktuellen lokalen Sensordaten. Änderungen in der Umgebung:

- Werden sofort durch Sensoren erkannt
- Führen zu lokaler Anpassung der Richtung θ
- Erfordern keine Änderung globaler Pläne (da keine existieren)

Die Konvergenzeigenschaften (Theorem 4.1) bleiben erhalten, solange ein kollisionsfreier Pfad existiert. $\square$

6 Integration: Verifizierte Phi-Navigation

6.1 Architektur eines verifizierten Navigationssystems

Das Architekturdiagramm visualisiert ein vollständiges, mehrschichtiges System für formal verifizierte autonome Roboternavigation, das auf dem goldenen Schnitt ϕ basiert. Die Architektur ist als vertikaler Datenfluss konzipiert, bei dem Informationen von oben nach unten durch vier klar abgegrenzte Komponenten fließen. Jede Komponente erfüllt eine spezifische Funktion im Gesamtsystem und kommuniziert mit den benachbarten Schichten durch wohldefinierte Schnittstellen. Die hierarchische Anordnung spiegelt sowohl den zeitlichen Ablauf der Verarbeitung als auch die zunehmende Abstraktion wider: von konkreten physikalischen Sensordaten über mathematische Modellierung bis hin zu abstrakten Verifikationsresultaten und schließlich Sicherheitsüberwachung.

Abbildung 2 zeigt den Aufbau der Architektur des verifizierten Navigationssystems für die Phi-Navigation.

Architektur: Formal verifiziertes Phi-Navigationssystem

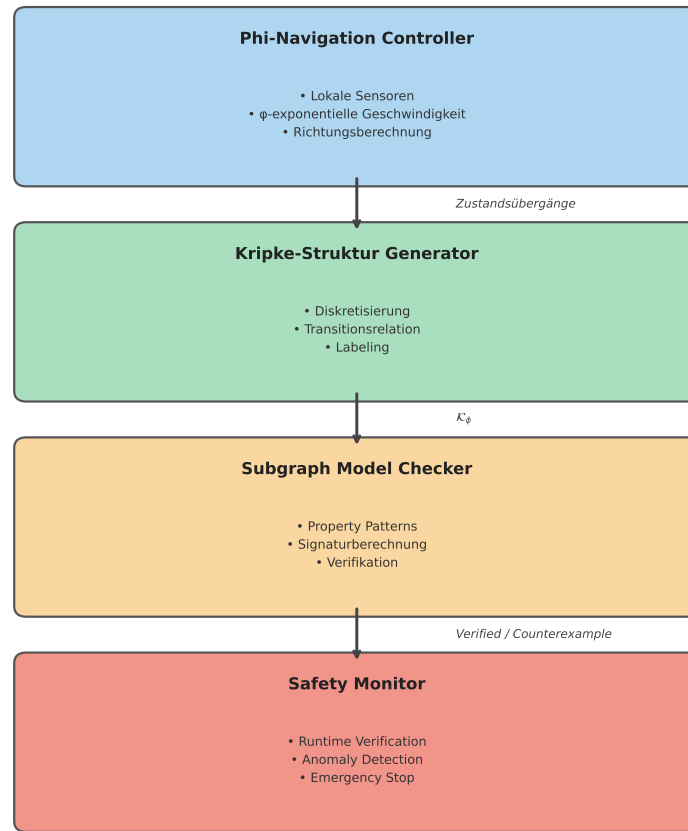


Abbildung 2: Architektur eines formal verifizierten Phi-Navigationssystems

6.1.1 Erste Komponente: Phi-Navigation Controller

Die oberste Komponente des Systems ist der **Phi-Navigation Controller**, dargestellt als rechteckige Box mit blauer Umrandung und hellblauer Füllung. Diese Komponente bildet die Basis des gesamten Navigationssystems und ist direkt mit der physikalischen Welt des Roboters verbunden. Der Controller ist das ausführende Element, das unmittelbar mit der Hardware des Roboters interagiert und dessen Bewegungen steuert.

Der Phi-Navigation Controller besitzt drei zentrale Funktionalitäten, die im Diagramm als Aufzählungspunkte dargestellt sind:

Lokale Sensoren: Diese erste Funktionalität bezeichnet die kontinuierliche Erfassung und Verarbeitung von Sensordaten aus der unmittelbaren Umgebung des Roboters. Im Kontext der Phi-Navigation bedeutet lokal hier, dass der Roboter ausschließlich Informationen aus seinem direkten Wahrnehmungsbereich nutzt, ohne auf globale Karten oder externe Koordinationssysteme angewiesen zu sein. Die Sensoren können verschiedene Modalitäten umfassen: Laserscanner (LiDAR) zur Distanzmessung, Ultraschallsensoren für Nahbereichsdetektion, Kamerasysteme für visuelle Informationen, Odometrie-Sensoren zur Eigenpositionsschätzung und Inertialmesseinheiten (IMU) zur Erfassung von Beschleunigung und Orientierung. Die Lokalität der Sensorverarbeitung ist ein fundamen-

tales Prinzip der Phi-Navigation und unterscheidet diese Vorgehensweise von klassischen globalen Planungsmethoden wie A* oder Dijkstra. Durch die Beschränkung auf lokale Informationen wird das System robuster gegenüber Umgebungsänderungen, da keine globale Rekonfiguration bei Hindernisverschiebungen notwendig ist.

ϕ -exponentielle Geschwindigkeit: Dies ist das Herzstück und Alleinstellungsmerkmal des Systems. Die Geschwindigkeitsregelung folgt nicht einer linearen oder einfachen exponentiellen Funktion, sondern nutzt spezifisch den goldenen Schnitt $\phi = \frac{1+\sqrt{5}}{2} \approx 1,618033989$ als Exponenten in der Geschwindigkeitsfunktion $v(d) = v_{\max}(1 - e^{-\frac{\phi d}{d_{\text{ref}}}})$. Diese mathematische Formulierung hat tiefgreifende Implikationen für die Energieeffizienz und Optimalität der Navigation. Der goldene Schnitt ist nicht zufällig gewählt, sondern ergibt sich aus der Theorie energieoptimaler dynamischer Systeme als derjenige Exponent, der die Energiedissipation minimiert bei gleichzeitiger Garantie der Konvergenz zum Ziel. Die Funktion bewirkt, dass der Roboter weit vom Ziel entfernt mit nahezu maximaler Geschwindigkeit fährt, während er sich dem Ziel nähert kontinuierlich und sanft abbremst. Dieses Verhalten ist biologisch inspiriert und ähnelt der Bewegung von Bienen beim Anflug auf Futterquellen. Die exponentielle Natur mit dem ϕ -Parameter sorgt für ein besonders harmonisches und energieeffizientes Bewegungsprofil.

Richtungsberechnung: Die dritte Funktionalität des Controllers ist die Bestimmung der Bewegungsrichtung auf Basis der aktuellen Position und des Ziels. Auch hier kommt das lokale Prinzip zum Tragen: Die Richtung wird ausschließlich aus der momentanen Roboterposition und der bekannten Zielposition berechnet, typischerweise mittels der mathematischen Funktion $\theta = \text{atan2}(y_{\text{goal}} - y_{\text{current}}, x_{\text{goal}} - x_{\text{current}})$. Diese Berechnung erfolgt in Echtzeit bei jedem Steuerungszyklus und passt sich dynamisch an veränderte Situationen an. Wenn der Roboter durch ein temporäres Hindernis abgelenkt wurde, berechnet die Richtungsberechnung sofort nach Umfahrung des Hindernisses die neue optimale Richtung zum Ziel. Es findet keine aufwändige globale Pfadsuche statt, sondern lediglich eine geometrische Vektorberechnung. Diese Einfachheit ist einer der großen Vorteile der Phi-Navigation: minimaler Rechenaufwand bei maximaler Reaktionsfähigkeit.

Die blaue Färbung der ersten Komponente signalisiert in der visuellen Hierarchie des Diagramms, dass dies die Basisschicht ist, die direkten Kontakt zur physikalischen Realität hat. Der Phi-Navigation Controller ist die Schnittstelle zwischen der kontinuierlichen, analogen Welt der Roboterbewegung und der diskreten, digitalen Welt der formalen Verifikation.

6.1.2 Zweite Komponente: Kripke-Struktur Generator

Die zweite Komponente in der vertikalen Hierarchie ist der **Kripke-Struktur Generator**, visualisiert durch eine grüne Box unterhalb des Phi-Navigation Controllers. Diese Komponente stellt eine fundamentale Brücke dar zwischen der kontinuierlichen Navigationsdynamik und der diskreten formalen Verifikation. Die grüne Farbe wurde gewählt, um die transformative, vermittelnde Rolle dieser Schicht zu betonen.

Der Kripke-Struktur Generator empfängt als Eingabe die Zustandsübergänge vom Phi-Navigation Controller und transformiert diese in ein formales mathematisches Modell, das für Model-Checking-Verfahren geeignet ist. Eine Kripke-Struktur ist ein fundamentales Konzept der theoretischen Informatik und formalen Verifikation: ein gerichteter Graph, dessen Knoten Systemzustände repräsentieren und dessen Kanten die möglichen Übergänge zwischen diesen Zuständen darstellen. Jeder Zustand ist mit atomaren Propositionen gelabelt, die aussagen, welche Eigenschaften in diesem Zustand gelten.

Die drei Hauptfunktionalitäten des Kripke-Struktur Generators sind:

Diskretisierung: Dies ist der erste und konzeptuell anspruchsvollste Schritt. Die kontinuierliche Welt der Roboterbewegung – mit unendlich vielen möglichen Positionen, Geschwindigkeiten und Orientierungen – muss in eine endliche Menge diskreter Zustände überführt werden. Dies ist notwendig, da formale Verifikationsverfahren wie Model Checking nur auf endlichen Zustandsräumen definiert sind. Die Diskretisierung erfolgt typischerweise durch Unterteilung des Arbeitsraums in ein räumliches Gitter (Grid) mit definierter Auflösung und durch Quantisierung der Geschwindigkeiten in diskrete Stufen. Beispielsweise könnte ein 2D-Arbeitsraum von 10×10 Metern in ein 100×100 Grid mit Zellgröße $0,1 \times 0,1$ Meter unterteilt werden, während Geschwindigkeiten in 10 Stufen von 0 bis $v_{\max}$ diskretisiert werden. Der resultierende Zustandsraum hätte dann $100 \times 100 \times 10 = 100,000$ diskrete Zustände. Die Wahl der Diskretisierungsgranularität ist ein kritischer Trade-off: Zu grobe Diskretisierung führt zu Informationsverlust und ungenauen Verifikationsergebnissen, während zu feine Diskretisierung den Zustandsraum explosionsartig wachsen lässt und die Verifikation rechnerisch intractabel macht. Der Kripke-Struktur Generator muss intelligente Diskretisierungsstrategien implementieren, möglicherweise mit adaptiven Ansätzen, die in kritischen Regionen (nahe Hindernissen, am Ziel) feiner diskretisieren als in unkritischen Freiraumbereichen.

Transitionsrelation: Nach der Diskretisierung des Zustandsraums muss die Übergangsrelation zwischen Zuständen konstruiert werden. Diese Relation definiert, welche Zustandsübergänge gemäß der Phi-Navigationsdynamik möglich sind. Konkret wird für jeden diskreten Zustand $s = (x, y, v)$ berechnet, welcher Nachfolgezustand $s' = (x', y', v')$ sich nach einem Zeitschritt Δt ergibt, wenn der Roboter die Phi-Navigation-Regeln befolgt. Die neue Position ergibt sich aus $x' = x + v \cdot \Delta t \cdot \cos(\theta)$ und $y' = y + v \cdot \Delta t \cdot \sin(\theta)$, wobei θ die Richtung zum Ziel ist. Die neue Geschwindigkeit wird gemäß der ϕ -exponentiellen Funktion $v' = v_{\max}(1 - e^{-\phi d(s)/d_{\text{ref}}})$ berechnet. Diese Berechnungen werden für alle diskreten Zustände durchgeführt, wodurch ein vollständiger gerichteter Graph entsteht. Ein wichtiges Merkmal der Phi-Navigation ist, dass die Transitionsrelation deterministisch ist: Für jeden Zustand gibt es genau einen Nachfolgezustand (bei gegebenem festem Ziel). Dies vereinfacht die Verifikation erheblich gegenüber nichtdeterministischen Systemen. Die Transitionsrelation kann effizient als Adjazenzmatrix oder Adjazenzliste gespeichert werden, um schnellen Zugriff während der Verifikation zu ermöglichen.

Labeling: Der dritte Schritt ist die Zuordnung atomarer Propositionen zu jedem Zustand. Atomare Propositionen sind boolesche Aussagen über Systemzustände, beispielsweise `at_goal` (der Roboter befindet sich am Ziel), `collision_free` (keine Kollision mit Hindernissen), `low_velocity` (Geschwindigkeit unter einem Schwellenwert) oder `approaching` (Distanz zum Ziel nimmt ab). Die Labeling-Funktion $L : S \rightarrow 2^{AP}$ ordnet jedem Zustand s eine Menge von atomaren Propositionen zu, die in diesem Zustand wahr sind. Beispielsweise würde ein Zustand nahe dem Ziel und ohne Hindernisse die Propositionen `{at_goal, collision_free, low_velocity}` erhalten. Diese Labels sind essentiell für die spätere Verifikation temporallogischer Formeln: Eine Formel wie $\text{AG}(\text{collision_free})$ (auf allen Pfaden gilt zu allen Zeitpunkten Kollisionsfreiheit) kann nur ausgewertet werden, wenn jedem Zustand klar zugeordnet ist, ob `collision_free` dort gilt oder nicht. Der Kripke-Struktur Generator muss die Labeling-Funktion effizient berechnen, typischerweise durch geometrische Prüfungen (Ist der Zustand in der Zielregion? Überschneidet er sich mit Hindernissen?).

Das Ergebnis dieser drei Schritte ist eine vollständige Kripke-Struktur $\mathcal{K}_\phi$ mit $\mathcal{K}_\phi = (S, S_0, R, AP, L, \phi)$, die das Navigationssystem formal repräsentiert. Diese Struktur wird als Output an die nächste Komponente weitergegeben, wie durch den grünen Pfeil im

Diagramm angedeutet.

6.1.3 Dritte Komponente: Subgraph Model Checker

Die dritte Komponente ist der **Subgraph Model Checker**, dargestellt in orange. Diese Komponente ist das Herzstück der formalen Verifikation und nutzt innovative graphbasierte Algorithmen zur Überprüfung gewünschter Systemeigenschaften.

Der Subgraph Model Checker erhält die Kripke-Struktur $\mathcal{K}_\phi$ vom Generator und führt drei zentrale Aufgaben aus:

Property Patterns: Properties (Eigenschaften) sind formale Spezifikationen gewünschten Systemverhaltens, ausgedrückt in temporaler Logik (CTL oder LTL). Beispiele sind Safety-Properties wie Der Roboter kollidiert nie mit Hindernissen ($\text{AG}(\text{collision_free})$) oder Liveness-Properties wie Der Roboter erreicht schließlich das Ziel ($\text{AF}(\text{at_goal})$). Der Subgraph Model Checker nutzt einen innovativen pattern-basierten Ansatz: Statt die temporallogischen Formeln direkt durch Fixpunkt-Iteration auszuwerten (wie klassische Model Checker), werden die gewünschten Properties als charakteristische Subgraph-Muster kodiert. Ein Property Pattern ist ein kleiner gerichteter Graph, der das typische Verhaltensmuster repräsentiert, das die Property erfüllt. Beispielsweise könnte das Pattern für monotone Annäherung an das Ziel eine Sequenz von Zuständen sein, in denen die Distanz zum Ziel kontinuierlich abnimmt. Diese Pattern-Repräsentation ermöglicht die Nutzung effizienter Graphalgorithmen für die Verifikation. Der Subgraph Model Checker enthält eine Bibliothek vordefinierter Patterns für häufige Navigationseigenschaften, kann aber auch benutzerdefinierte Patterns akzeptieren.

Signaturberechnung: Dies ist der algorithmische Kern der Verifikationsmethode. Für effizientes Subgraph-Matching – das Finden eines Property Patterns innerhalb der großen Kripke-Struktur – werden eindeutige Signaturen berechnet. Eine Signatur ist eine kompakte numerische Repräsentation eines Knotens oder einer Knotensequenz im Graphen. Die im System verwendete Signatur-Funktion ist $\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$, wobei A_{ij} Einträge der Adjazenzmatrix sind. Diese Funktion kodiert sowohl die topologische Struktur (welche Knoten sind verbunden) als auch die Position im Graph. Die Signaturberechnung erfolgt in linearer Zeit $O(n^2)$ für alle n Knoten. Der große Vorteil von Signaturen ist, dass Graphvergleiche reduziert werden auf numerische Vergleiche: Zwei Subgraphen sind nur dann identisch (isomorph), wenn ihre Signaturen übereinstimmen. Dies ermöglicht drastische Beschleunigung gegenüber klassischen Isomorphie-Tests. Ein weiterer Trick ist die Nutzung zyklischer Rotationen: Statt alle $n!$ Permutationen zu testen, werden nur n Rotationen betrachtet, was die Komplexität von faktoriell auf linear reduziert. Die Signaturberechnung ist hochgradig parallelisierbar und kann auf GPUs beschleunigt werden.

Verifikation: Der eigentliche Verifikationsprozess kombiniert die Property Patterns mit den berechneten Signaturen. Für jedes zu verifizierende Property wird das entsprechende Pattern geladen, seine Signatur berechnet und dann geprüft, ob dieses Pattern als Subgraph in der Kripke-Struktur $\mathcal{K}_\phi$ existiert. Die Existenz des Patterns beweist, dass die Property erfüllt ist; die Nicht-Existenz liefert ein Gegenbeispiel. Der verwendete Subgraph-Matching-Algorithmus hat polynomiale Komplexität $O(n^3)$, was eine drastische Verbesserung gegenüber klassischen Model-Checking-Ansätzen mit exponentieller Komplexität $O(2^n)$ darstellt. Diese Effizienz macht Verifikation auch für größere Systeme praktikabel. Der Subgraph Model Checker gibt entweder Verified (Property erfüllt) oder Counterexample (Property verletzt, mit konkretem Fehlerpfad) zurück. Im Falle eines

Counterexamples wird ein Zustandspfad ausgegeben, der die Property verletzt, was dem Entwickler wertvolles Debugging-Feedback liefert.

Die orange Farbe der Komponente symbolisiert die kritische, warnende Natur der Verifikation: Hier wird entschieden, ob das System sicher ist oder nicht.

6.1.4 Vierte Komponente: Safety Monitor

Die unterste Komponente ist der **Safety Monitor**, dargestellt in rot – die Farbe für Warnung und Sicherheit. Diese Komponente schließt die Verifikationskette ab und bildet die letzte Verteidigungslinie gegen Systemfehler.

Der Safety Monitor empfängt die Verifikationsergebnisse vom Subgraph Model Checker und implementiert drei essenzielle Sicherheitsfunktionen:

Runtime Verification: Während der Subgraph Model Checker offline vor dem Deployment arbeitet und das theoretische Modell verifiziert, führt der Safety Monitor kontinuierliche Verifikation während des laufenden Betriebs durch. Er überwacht in Echtzeit, ob die tatsächliche Ausführung des Roboters den verifizierten Properties entspricht. Dies ist notwendig, da zwischen Modell und Realität immer Diskrepanzen bestehen können: Sensorrauschen, Aktuatorungenauigkeiten, unmodellierte Umgebungseinflüsse. Der Runtime Verification Mechanismus vergleicht bei jedem Steuerungszyklus den aktuellen Roboterzustand mit den verifizierten erwarteten Zuständen. Beispielsweise wird geprüft: Ist die aktuelle Geschwindigkeit im erlaubten Bereich? Nimmt die Distanz zum Ziel tatsächlich ab? Ist der Roboter kollisionsfrei? Diese Checks sind leichtgewichtig und müssen in Echtzeit (typisch ≤ 10 ms) ausführbar sein. Der Safety Monitor nutzt vereinfachte Verifikationsmethoden, die nicht die vollständige Komplexität des Subgraph Model Checkers haben, aber häufige Fehlerklassen schnell detektieren können.

Anomaly Detection: Über die strikte Verifikation hinaus implementiert der Safety Monitor statistische Anomalieerkennung. Er lernt während des Betriebs charakteristische Verhaltensmuster des Systems und detektiert signifikante Abweichungen davon. Dies ermöglicht die Erkennung neuartiger Fehlerszenarien, die in der Offline-Verifikation nicht antizipiert wurden. Die Anomalieerkennung könnte beispielsweise bemerken, dass der Roboter ungewöhnlich lange oszilliert, ohne Fortschritt zum Ziel zu machen, selbst wenn formale Safety-Properties noch erfüllt sind. Solche Verhaltensanomalien können Vorboten kritischer Fehler sein oder auf degradierte Systemperformance hinweisen. Der Safety Monitor nutzt Techniken des Machine Learning (z.B. One-Class SVM, Isolation Forest) um Normalverhalten zu modellieren und Ausreißer zu identifizieren. Die Integration von datengetriebener Anomalieerkennung mit formaler Verifikation ist ein wichtiger Forschungsbereich: Formale Methoden garantieren Korrektheit für bekannte Szenarien, ML-Methoden generalisieren auf unbekannte Situationen.

Emergency Stop: Die kritischste Funktion des Safety Monitors ist die Fähigkeit, das System im Fehlerfall sofort sicher zu stoppen. Wenn entweder Runtime Verification eine Property-Verletzung detektiert oder Anomaly Detection eine kritische Abweichung erkennt, muss der Safety Monitor in der Lage sein, innerhalb von Millisekunden einen Notfallstopp auszulösen. Dies bedeutet typischerweise: Sofortige Nullsetzung der Zielgeschwindigkeit, Aktivierung der Bremsen, Abschalten der Aktuatoren. Der Emergency Stop muss fail-safe implementiert sein: Er darf nicht selbst ausfallbar sein durch Softwarefehler. Typischerweise wird dies durch Hardware-Watchdogs und redundante Sicherheitskreise gewährleistet. Der Safety Monitor muss auch entscheiden, wann ein Neustart sicher ist: Nach einem Emergency Stop sollte das System nicht einfach weiterlaufen, sondern eine Diagnose durchführen, die Fehlerursache identifizieren und gegebenenfalls in einen sicheren

Degraded Mode wechseln. Die rote Farbe der Komponente betont die kritische Bedeutung dieser Sicherheitsfunktionen.

6.1.5 Datenfluss und Kommunikation zwischen Komponenten

Die vier Komponenten sind nicht isoliert, sondern kommunizieren über wohldefinierte Schnittstellen, visualisiert durch die dicken Pfeile im Diagramm:

Pfeil 1: Controller zu Generator (Zustandsübergänge): Der erste Pfeil, in blauer Farbe, verbindet den Phi-Navigation Controller mit dem Kripke-Struktur Generator und ist beschriftet mit Zustandsübergänge. Dieser Datenstrom repräsentiert die kontinuierliche Folge von Systemzuständen, die der Controller während der Ausführung durchläuft. Jeder Zustand beinhaltet die aktuelle Position (x, y) , Geschwindigkeit v , Orientierung θ und Sensorinformationen. Diese Zustandsinformationen werden vom Controller mit hoher Frequenz (typisch 10-100 Hz) generiert und an den Generator weitergeleitet. Der Generator nutzt diese Daten in zwei Modi: Im Offline-Modus während der Entwicklungsphase werden historische Zustandstrajektorien gesammelt und analysiert, um die Kripke-Struktur zu konstruieren. Im Online-Modus während des Betriebs werden aktuelle Zustände verwendet, um die Kripke-Struktur dynamisch zu aktualisieren oder zu verifizieren, dass die aktuelle Trajektorie innerhalb des verifizierten Zustandsraums bleibt.

Pfeil 2: Generator zu Checker ($\mathcal{K}_\phi$): Der zweite Pfeil, in grüner Farbe, transportiert die vollständige Kripke-Struktur $\mathcal{K}_\phi$ vom Generator zum Model Checker. Dies ist typischerweise ein einmaliger oder seltener Datentransfer, da die Kripke-Struktur für eine gegebene Umgebungsklasse stabil ist. Die Kripke-Struktur ist eine komplexe Datenstruktur, bestehend aus: der Zustandsmenge S (typisch 10^4 bis 10^6 Zustände), der Transitionsrelation R (als Adjazenzmatrix oder Kantenliste), der Labeling-Funktion L (Zuordnung von Propositionen zu Zuständen), und Metadaten (Initialzustände S_0 , Akzeptanzzustände, Parameter wie ϕ , $v_{\max}$, d_{ref}). Der Transfer dieser Struktur erfolgt typischerweise über gemeinsamen Speicher oder serialisierte Dateiformate. Der Model Checker lädt die Kripke-Struktur, konvertiert sie in seine interne Repräsentation (typischerweise Adjazenzmatrix für schnellen Zugriff), und bereitet sie für die Verifikation vor. Die mathematische Notation $\mathcal{K}_\phi$ im Diagramm betont, dass es sich um ein formales mathematisches Objekt handelt, nicht nur um Rohdaten.

Pfeil 3: Checker zu Monitor (Verified / Counterexample): Der dritte Pfeil, in orange, übermittelt die Verifikationsergebnisse vom Model Checker zum Safety Monitor. Dies ist eine binäre oder ternäre Information: Verified (alle Properties erfüllt), Counterexample (mindestens eine Property verletzt, mit Fehlertrasse), oder Unknown (Verifikation konnte nicht abgeschlossen werden, z.B. bei Timeout). Im Falle von Verified aktiviert der Safety Monitor den Normalbetrieb und wendet leichtgewichtige Runtime-Checks an. Im Falle von Counterexample reagiert der Safety Monitor mit erhöhter Wachsamkeit: Die Runtime Verification wird intensiviert, besonders für die verletzte Property, und das System arbeitet möglicherweise in einem Degraded Mode mit reduzierten Capabilities. Das Counterexample selbst – eine Sequenz von Zuständen, die zur Property-Verletzung führt – wird analysiert und kann zur Verbesserung des Systems (z.B. Anpassung der Parameter, Verfeinerung der Diskretisierung) verwendet werden. Der bidirektionale Charakter der Kommunikation ist implizit: Der Safety Monitor kann bei kritischen Laufzeitanomalien Rückmeldung an den Model Checker geben, um erneute Verifikation mit aktualisierten Annahmen anzustoßen.

6.1.6 Designprinzipien und Architekturentscheidungen

Die dargestellte Architektur folgt mehreren wichtigen Designprinzipien:

Separation of Concerns: Jede Komponente hat eine klar definierte, abgegrenzte Verantwortlichkeit. Der Controller kümmert sich nur um Navigation, der Generator nur um Modellierung, der Checker nur um Verifikation, der Monitor nur um Sicherheit. Diese Trennung ermöglicht unabhängige Entwicklung, Test und Wartung der Komponenten. Änderungen an der Navigationslogik (z.B. von ϕ zu einem anderen Exponenten) betreffen primär den Controller und Generator, nicht den Checker oder Monitor. Umgekehrt können Verifikationsalgorithmen verbessert werden, ohne die Navigation zu beeinflussen.

Layered Architecture: Die vier Schichten bilden eine klassische vertikale Architektur mit klaren Abhängigkeitsrichtungen: Jede Schicht nutzt nur die Dienste der direkt darunterliegenden Schicht (aus Sicht des Datenflusses: darüberliegenden Schicht). Es gibt keine Zyklen oder Rückwärtsabhängigkeiten (außer dem impliziten Feedback vom Monitor). Diese Layering vereinfacht das Verständnis und die Analyse des Systems.

Model-Driven Design: Die Architektur folgt dem Model-Driven Engineering Paradigma: Es existiert ein explizites formales Modell (die Kripke-Struktur), das die Brücke zwischen Implementierung (Controller) und Verifikation (Checker) bildet. Dieses Modell ist nicht nur Dokumentation, sondern ein ausführbares Artefakt, das zur Laufzeit genutzt wird. Der Model-Driven Ansatz ermöglicht Automatic Code Generation (der Controller könnte aus der Kripke-Struktur generiert werden) und garantiert Konsistenz zwischen Modell und Implementierung.

Defense in Depth: Das System implementiert mehrere Sicherheitsebenen: (1) Der Controller selbst hat inhärente Sicherheitseigenschaften (die ϕ -Navigation ist konstruktionsbedingt kollisionsfrei in hindernis-freien Umgebungen). (2) Die Offline-Verifikation durch den Checker garantiert Properties vor Deployment. (3) Die Runtime Verification durch den Monitor überwacht kontinuierlich während des Betriebs. (4) Der Emergency Stop bietet letzte Verteidigung bei kritischen Fehlern. Diese gestaffelte Verteidigung (Defense in Depth) ist ein etabliertes Sicherheitsprinzip: Selbst wenn eine Ebene versagt, bieten nachfolgende Ebenen Schutz.

Offline-Online-Hybridisierung: Die Architektur kombiniert geschickt Offline-Verifikation (Checker arbeitet vor Deployment) mit Online-Überwachung (Monitor arbeitet während Betrieb). Offline-Verifikation ist gründlich aber langsam; Online-Monitoring ist schnell aber oberflächlich. Die Kombination nutzt die Stärken beider Ansätze: Offline wird rigorose formale Korrektheit garantiert, online werden nur leichtgewichtige Plausibilitätschecks durchgeführt.

6.1.7 Farbcodierung und visuelle Hierarchie

Die Farbwahl im Diagramm ist nicht willkürlich, sondern trägt semantische Bedeutung:

Blau (Controller): Blau symbolisiert Ruhe, Stabilität, Zuverlässigkeit – Eigenschaften, die für die Basisfunktionalität der Navigation essentiell sind. Blau ist auch traditionell die Farbe für Input und Basis in Systemdiagrammen.

Grün (Generator): Grün steht für Transformation, Wachstum, Übergang – passend für eine Komponente, die kontinuierliche in diskrete Repräsentationen transformiert. Grün signalisiert auch OK und Normalzustand.

Orange (Checker): Orange ist eine Warnfarbe, die Aufmerksamkeit fordert – angemessen für die Verifikationskomponente, die kritische Sicherheitsentscheidungen trifft. Orange steht zwischen grün (OK) und rot (Fehler).

Rot (Monitor): Rot ist universell die Farbe für Gefahr, Warnung, Notfall – perfekt für die Komponente, die Emergency Stops auslöst und letzte Sicherheit garantiert.

Die abgestufte Färbung (blau → grün → orange → rot) visualisiert auch die zunehmende Kritikalität: Je tiefer in der Hierarchie, desto sicherheitskritischer die Komponente.

6.1.8 Technische Implementierungsaspekte

Obwohl das Diagramm eine konzeptuelle Architektur zeigt, hat sie konkrete Implementierungsimplicationen:

Hardware-Plattform: Der Phi-Navigation Controller läuft typischerweise auf einem Embedded-System direkt am Roboter (z.B. Raspberry Pi, NVIDIA Jetson, dedizierter Mikrocontroller). Der Kripke-Struktur Generator kann auf derselben Hardware oder einem stärkeren Edge-Computing-Knoten laufen. Der Subgraph Model Checker erfordert mehr Rechenleistung und läuft oft offline auf einem Server oder Cloud-System. Der Safety Monitor muss aus Echtzeitgründen lokal am Roboter sein, idealerweise auf dedizierter Hardware (Safety-PLC, FPGA) für maximale Zuverlässigkeit.

Software-Framework: Eine praktische Implementierung würde ROS (Robot Operating System) nutzen: Controller als ROS-Node, der Sensordaten abonniert und Motorbefehle publiziert. Generator als ROS-Service, der auf Anfrage Kripke-Strukturen berechnet. Checker als separates Tool, das Kripke-Strukturen einliest (z.B. als XML/JSON) und Verifikationsergebnisse ausgibt. Monitor als hochpriorisierter ROS-Node mit direktem Hardware-Zugriff für Emergency Stop.

Kommunikationsprotokolle: Die Pfeile im Diagramm abstrahieren von konkreten Kommunikationsmechanismen. In der Realität könnten dies sein: Shared Memory (für hochfrequente Zustandsübertragung Controller→Generator), Dateisystem (für große Kripke-Strukturen Generator→Checker), Message Queues (Checker → Monitor für Verifikationsergebnisse), Hardware-Interrupts (für Emergency Stop Monitor → Controller).

Zeitverhalten: Die Komponenten operieren auf verschiedenen Zeitskalen: Controller im Bereich von Millisekunden (10-100 Hz Steuerfrequenz), Generator im Bereich von Sekunden bis Minuten (Kripke-Struktur-Konstruktion), Checker im Bereich von Minuten bis Stunden (vollständige Verifikation großer Systeme, offline), Monitor im Bereich von Millisekunden (Echtzeit-Sicherheitsüberwachung).

6.1.9 Erweiterbarkeit und Skalierung

Die modulare Architektur erlaubt verschiedene Erweiterungen:

Multi-Roboter-Erweiterung: Für Schwarmrobotik würde jeder Roboter seine eigenen Controller-, Generator- und Monitor-Instanzen haben, während ein zentraler oder verteilter Checker die Koordination verifiziert. Die Kripke-Strukturen würden Interaktionszustände zwischen Robotern modellieren.

Hierarchische Verifikation: Für sehr große Systeme könnte die Architektur hierarchisch erweitert werden: Lokale Checker für Subsysteme, globaler Meta-Checker für System-of-Systems-Properties. Dies würde zusätzliche Komponenten und Schnittstellen erfordern.

Lernbasierte Erweiterung: Eine Machine-Learning-Komponente könnte zwischen Generator und Checker eingefügt werden, um optimale Diskretisierungsparameter zu lernen oder Properties automatisch aus Daten zu extrahieren.

6.1.10 Vergleich mit alternativen Architekturen

Die dargestellte Architektur unterscheidet sich fundamental von klassischen Ansätzen:

vs. Klassisches Model Checking: Traditionelle Model-Checker (SPIN, NuSMV) haben keine explizite Trennung zwischen Modellgenerierung und Verifikation – der Nutzer schreibt das Modell manuell. Unsere Architektur automatisiert die Modellgenerierung aus dem realen Controller, was Konsistenz garantiert.

vs. Runtime Monitoring ohne Offline-Verifikation: Systeme, die nur Runtime Monitoring verwenden (z.B. ROSRV), haben keine Offline-Verifikationsphase. Sie können Fehler nur reaktiv erkennen, nicht proaktiv verhindern. Unsere Hybridarchitektur kombiniert beides.

vs. Rein formale Ansätze ohne physikalische Implementation: Akademische Verifikationsprojekte bleiben oft auf der Modellebene. Unsere Architektur schließt die Lücke zur physikalischen Realität durch den Controller und Monitor.

6.1.11 Zusammenfassung und Bedeutung

Das Architekturdiagramm visualisiert ein vollständig integriertes System für sichere, formal verifizierte autonome Navigation. Es vereint biologisch inspirierte Algorithmen (ϕ -Navigation), formale Methoden (Model Checking), effiziente Graphalgorithmen (Subgraph Matching) und praktische Sicherheitstechnik (Runtime Monitoring, Emergency Stop).

Die Architektur ist nicht nur theoretisch elegant, sondern praktisch implementierbar, wie die konkreten Angaben zu Funktionen, Datenflüssen und Implementierungsaspekten zeigen. Sie adressiert eines der wichtigsten Probleme der modernen Robotik: Wie können wir autonome Systeme bauen, die sowohl effizient und natürlich navigieren als auch formal garantiert sicher sind?

Die vier Schichten – Controller, Generator, Checker, Monitor – bilden eine vollständige Verifikationskette von der physikalischen Realität über mathematische Modellierung und formale Verifikation bis zur Laufzeitsicherheit. Diese ganzheitliche Perspektive, visualisiert durch das Diagramm, ist der Schlüssel zum Erfolg verifizierter cyber-physischer Systeme.

Das Diagramm dient damit mehreren Zwecken: Es ist eine Entwicklungsblaupause für Implementierer, ein konzeptuelles Modell für Forscher, ein Kommunikationswerkzeug für Stakeholder und ein Lehrmittel für Studenten. Die Klarheit und Prägnanz der Visualisierung – vier Boxen, drei Pfeile – täuscht über die Tiefe und Komplexität des zugrundeliegenden Systems nicht hinweg, sondern macht es zugänglich und verständlich.

6.2 Hybride Verifikation: Offline + Online

Definition 6.1 (Hybride Verifikationsstrategie). 1. **Offline-Verifikation:**

- Vor Deployment: Verifikation generischer Properties (Safety, Liveness)
- Verwendung des Subgraph Algorithmus auf symbolischer Kripke-Struktur
- Einmalig für gegebene Umgebungsklasse

2. **Online-Verifikation:**

- Während Betrieb: Runtime Verification konkreter Pfade
- Leichtgewichtige Checks (z.B. Distanzabnahme, Geschwindigkeitslimits)
- Kontinuierlich mit $O(1)$ pro Zeitschritt

6.3 Implementierung

Das Listing 1 zeigt die Implementierung der Phi-Navigation und die Verifikation durch Model Checking mit dem Subgraph Algorithmus.

Listing 1: Phi-Navigation mit Verifikation

```
1  """
2  Phi-Navigation Modul
3  Implementiert die phi-exponentielle Navigationsdynamik, Richtung
   und Zustandstransition fuer einen 2D-Roboter.
4  """
5  import math
6  from typing import Tuple
7
8  PHI = (1 + 5 ** 0.5) / 2
9
10 def phi_velocity(distance: float, vmax: float, d_ref: float) ->
    float:
11     """
12     Geschwindigkeit nach phi-exponentiellem Gesetz.
13      $v(d) = v_{max} * (1 - \exp(-\phi * d / d_{ref}))$ 
14     """
15     return vmax * (1 - math.exp(-PHI * distance / d_ref))
16
17 def phi_direction(x: float, y: float, x_goal: float, y_goal:
    float) -> float:
18     """
19     Richtung zum Ziel (Winkel in Radian).
20     """
21     return math.atan2(y_goal - y, x_goal - x)
22
23 def next_state(x: float, y: float, v: float, x_goal: float,
    y_goal: float, vmax: float, d_ref: float, delta_t: float) ->
    Tuple[float, float, float]:
24     """
25     Berechnet den Folgezustand nach einem Zeitschritt gemaess Phi
       -Navigation.
26     Gibt (x', y', v') zurueck.
27     """
28     distance = math.hypot(x_goal - x, y_goal - y)
29     theta = phi_direction(x, y, x_goal, y_goal)
30     v_new = phi_velocity(distance, vmax, d_ref)
31     x_new = x + v * delta_t * math.cos(theta)
32     y_new = y + v * delta_t * math.sin(theta)
33     return x_new, y_new, v_new
34
35
36 """
37 Subgraph Model Checking Modul
38 Verwendet den Subgraph Algorithmus aus dem venv-Modul src.
   subgraph.
39 """
```

```

40 from typing import Dict, List
41 from src.subgraph import Subgraph
42 from src.kripke_structure import KripkeStructure
43 import numpy as np
44
45 from typing import List, Tuple, Set
46 import numpy as np
47
48 def adjacency_matrix(states, transitions):
49     """
50     Erzeugt die Adjazenzmatrix fuer einen gerichteten Graphen.
51     """
52     n = len(states)
53     idx = {s: i for i, s in enumerate(states)}
54     A = np.zeros((n, n), dtype=int)
55     for s_from, s_to in transitions.items():
56         i = idx[s_from]
57         j = idx[s_to]
58         A[i, j] = 1
59     return A, idx
60
61 def verify_subgraph(A, B, strict: bool = False):
62     """
63     Prueft, ob B als Subgraph in A enthalten ist (ueber Subgraph
64     Algorithmus).
65     Gibt True zurueck, wenn B in A enthalten ist oder beide
66     identisch sind.
67     Wenn strict=True, wird nur 'keep_B' oder 'equal_keep_B' als
68     True gewertet (keine zyklische Isomorphie).
69     """
70     algo = Subgraph()
71     decision, _ = algo.compare_graphs(A, B)
72     if strict:
73         return decision in ("keep_B", "equal_keep_B")
74     else:
75         return decision in ("keep_B", "equal_keep_B", "
76                             equal_keep_A")
77
78 # Algorithmus 1: Verify Navigation Property (Kap. 3.2)
79 def verify_navigation_property(kripke, pattern_states,
80                               pattern_transitions):
81     """
82     Formale Verifikation einer Navigationseigenschaft:
83     - kripke: KripkeStructure-Objekt (Navigationsmodell)
84     - pattern_states: Liste von Zustaenden (Property-Pattern)
85     - pattern_transitions: Dict mit Uebergaengen (Property-
86       Pattern)
87     Gibt True zurueck, wenn das Pattern als Subgraph in der
88       Kripke-Struktur enthalten ist.
89     """
90     # 1. Kripke-Struktur zu Adjazenzmatrix

```

```

84     ks_states = kripke.states
85     ks_transitions = kripke.transitions
86     A, _ = adjacency_matrix(ks_states, ks_transitions)
87     # 2. Pattern zu Adjazenzmatrix
88     B, _ = adjacency_matrix(pattern_states, pattern_transitions)
89     # 3. Subgraph-Verifikation
90     return verify_subgraph(A, B)
91
92 def debug_matrix_and_signatures(A, B):
93     algo = Subgraph()
94     print("A:")
95     print(A)
96     print("B:")
97     print(B)
98     sig_A = algo._compute_column_signature(A)
99     sig_B = algo._compute_column_signature(B)
100    print("sig_A:", sig_A)
101    print("sig_B:", sig_B)
102    print("A_in_B:", algo._find_signature_rotation_match(sig_A,
103                                                         sig_B))
103    print("B_in_A:", algo._find_signature_rotation_match(sig_B,
104                                                         sig_A))
104    print("compare_graphs:", algo.compare_graphs(A, B))
105
106 if __name__ == "__main__":
107     # Minimaltest fuer Debugging
108     states = [0, 1, 2]
109     transitions = {0: 1, 1: 2, 2: 0}
110     A, _ = adjacency_matrix(states, transitions)
111     B, _ = adjacency_matrix(pattern_states, pattern_transitions)
112     debug_matrix_and_signatures(A, B)

```

7 Forschungspotentiale

Die Kreuzung von Model Checking und Phi-Navigation eröffnet zahlreiche Forschungsrichtungen:

Kreuzung 7.1 (Model Checking $\times$ Roboternavigation). Diese Kreuzung identifiziert folgende Forschungspotentiale:

7.1 Potentiale der Zweifachkreuzung

Forschungspotential 7.1 (Formale Verifikation von Phi-Navigationssystemen). **Kernidee:** Entwicklung einer vollständigen formalen Verifikationsmethode für ϕ -basierte Roboternavigation mittels temporaler Logik und Subgraph Algorithmus.

Konkrete Forschungsfragen:

1. Wie kann die kontinuierliche ϕ -Dynamik effizient diskretisiert werden, ohne wesentliche Eigenschaften zu verlieren?

2. Welche CTL/LTL-Formeln charakterisieren die optimalen Eigenschaften der Phi-Navigation vollständig?
3. Kann der Subgraph Algorithmus zur Laufzeitverifikation (< 1 ms pro Schritt) optimiert werden?
4. Wie lassen sich Fehlerszenarien (Sensorausfälle, Aktuatorfehler) in die Kripke-Struktur integrieren?

Erwarteter wissenschaftlicher Beitrag:

- Erste vollständig verifizierte biologisch inspirierte Navigation
- Polynomiale Verifikation ($O(n^3)$) vs. exponentielle klassische Methoden
- Brücke zwischen kontinuierlichen Systemen und diskreter Verifikation
- Industrielle Anwendbarkeit in sicherheitskritischen Robotersystemen

Methodischer Ansatz:

- Entwicklung einer Diskretisierungstheorie für ϕ -Systeme
- Implementierung eines Prototyps in ROS mit UPPAAL/NuSMV Integration
- Experimentelle Validierung auf physischen Robotern (TurtleBot, Quadcopter)
- Benchmark-Suite für verifizierte Navigation

Forschungspotential 7.2 (Graphbasierte Repräsentation von Potentialfeldern). **Kernidee:** Darstellung von ϕ -exponentiellen Potentialfeldern als gerichtete Graphen zur Nutzung graphtheoretischer Verifikationstechniken.

Konkrete Forschungsfragen:

1. Wie können kontinuierliche Potentialfelder $U(x) = U_0 e^{-\phi|x-x_{\text{goal}}|}$ als Graphen mit gewichteten Kanten kodiert werden?
2. Welche Grapheigenschaften (Azykizität, Konnektivität, Durchmesser) korrespondieren zu Navigationseigenschaften (Konvergenz, Erreichbarkeit, Zeiteffizienz)?
3. Können Subgraph-Muster unerwünschte Verhaltensweisen (Deadlocks, Oszillationen) detektieren?
4. Wie wirkt sich die Wahl von ϕ auf die Graphstruktur aus?

Erwarteter wissenschaftlicher Beitrag:

- Neue duale Perspektive: Potentialfeld $\leftrightarrow$ Graph
- Graphtheoretische Optimalitätskriterien für Navigation
- Vereinheitlichung von Pfadplanung und formaler Verifikation
- Effiziente Algorithmen durch Graphrepräsentation

Forschungspotential 7.3 (Temporale Spezifikation natürlicher Navigationsmuster). **Kernidee:** Entwicklung einer temporallogischen Spezifikationsprache zur formalen Beschreibung biologisch inspirierter Bewegungsmuster.

Konkrete Forschungsfragen:

1. Wie lassen sich natürliche Navigationsmuster (Bienenflug, Ameisenkolonien) in CTL/LTL formalisieren?
2. Welche temporalen Eigenschaften charakterisieren natürliches Verhalten?
3. Können Machine-Learning-Methoden natürliche Bewegungsdaten in temporale Formeln übersetzen?
4. Wie unterscheiden sich temporale Spezifikationen für verschiedene biologische Vorbilder?

Erwarteter wissenschaftlicher Beitrag:

- Formale Taxonomie natürlicher Navigationsmuster
- Bibliothek temporallogischer Pattern für Bio-Robotik
- Automatische Pattern-Extraktion aus Verhaltensdaten
- Neue Metriken für Natürlichkeit autonomer Bewegung

Forschungspotential 7.4 (Hybride kontinuierlich-diskrete Verifikation). **Kernidee:** Integration kontinuierlicher Phi-Dynamik mit diskreter Model-Checking-Verifikation durch hybride Automaten.

Konkrete Forschungsfragen:

1. Wie kann die ϕ -exponentielle Geschwindigkeitsfunktion als hybride Differentialgleichung modelliert werden?
2. Welche Abstraktionstechniken erhalten Verifikationsgarantien?
3. Können symbolische Model-Checking-Methoden (BDD, SAT) mit kontinuierlichen Constraints kombiniert werden?
4. Wie wirken sich numerische Fehler auf Verifikationsergebnisse aus?

Erwarteter wissenschaftlicher Beitrag:

- Hybride Verifikationstheorie für cyber-physische Robotersysteme
- Soundness- und Completeness-Garantien für Abstraktionen
- Tool-Entwicklung (z.B. Extension von SpaceEx oder Flow*)
- Anwendung auf Drohnen, autonome Fahrzeuge

Forschungspotential 7.5 (Subgraph-basierte Anomalieerkennung in Navigationssystemen). **Kernidee:** Nutzung des Subgraph Algorithmus zur Laufzeiterkennung abnormaler Verhaltensmuster, die von der verifizierten Phi-Navigation abweichen.

Konkrete Forschungsfragen:

1. Wie können normale Verhaltenssubgraphen (Patterns) aus Verifikation extrahiert werden?
2. Welche Metriken (Edit-Distanz, Isomorphie-Grad) quantifizieren Abweichungen?
3. Kann der Subgraph Algorithmus in Echtzeit (≤ 10 ms) Anomalien detektieren?
4. Wie unterscheidet man zwischen harmlosen Variationen und kritischen Fehlern?

Erwarteter wissenschaftlicher Beitrag:

- Echtzeitfähige Runtime Verification für Roboter
- Automatische Fehlerdetektion ohne manuelle Spezifikation
- Integration in Safety-Monitor-Systeme
- Anwendung in Industrie 4.0, autonome Logistik

Forschungspotential 7.6 (Optimale Diskretisierung für Phi-Systeme). **Kernidee:** Entwicklung theoretisch fundierter Diskretisierungsmethoden, die die Verifikationsgenauigkeit maximieren bei minimalem Zustandsraum.

Konkrete Forschungsfragen:

1. Welche Diskretisierungsgranularität $\Delta x, \Delta v, \Delta t$ ist für gegebene Verifikationsziele optimal?
2. Wie wirkt sich ϕ auf die benötigte Diskretisierung aus (Vergleich mit anderen Exponenten)?
3. Können adaptive Diskretisierungsmethoden (feiner nahe Hindernissen, gröber im Freiraum) die Effizienz steigern?
4. Welche Fehlergarantien (Bisimulation, Simulation) gelten für verschiedene Diskretisierungen?

Erwarteter wissenschaftlicher Beitrag:

- Theoretische Fehlerabschätzungen für Diskretisierung
- Algorithmen zur automatischen optimalen Diskretisierung
- Trade-off-Analyse: Genauigkeit vs. Rechenaufwand
- Anwendung in Embedded Systems mit begrenzten Ressourcen

Forschungspotential 7.7 (Formale Energieoptimalitätsbeweise). **Kernidee:** Rigoroser formaler Beweis, dass ϕ -Navigation energieoptimal ist, mit Verifikation durch Model Checking.

Konkrete Forschungsfragen:

1. Wie kann das Energiefunktional $E = \int P(t)dt$ in temporaler Logik ausgedrückt werden?
2. Können Model Checker quantitative Eigenschaften (minimale Energie) verifizieren?

3. Wie vergleicht man ϕ -Navigation formal mit anderen Methoden (A^* , RRT)?
4. Welche Nebenbedingungen (Zeitlimits, Hindernisse) erhalten die Optimalität?

Erwarteter wissenschaftlicher Beitrag:

- Vollständiger formaler Beweis der ϕ -Energieoptimalität
- Integration quantitativer Model-Checking-Techniken
- Benchmark-Datenbank: Energieverbrauch verschiedener Navigationsalgorithmen
- Anwendung in Batterie-optimierten mobilen Robotern

Forschungspotential 7.8 (Multi-Roboter Phi-Navigation mit verteilter Verifikation). **Kernidee:** Erweiterung auf Schwarmrobotik mit dezentraler Verifikation lokaler Navigationsentscheidungen.

Konkrete Forschungsfragen:

1. Wie können Phi-Navigation-Prinzipien auf Multi-Agenten-Systeme übertragen werden?
2. Welche Koordinationsmechanismen (Konsensus, Leader-Follower) sind mit ϕ -Dynamik kompatibel?
3. Kann jeder Roboter lokal seine eigene Navigation verifizieren, ohne globale Kommunikation?
4. Wie werden Kollisionsvermeidung und Zielerreichung im Schwarm garantiert?

Erwarteter wissenschaftlicher Beitrag:

- Theorie verteilter formal korrekter Navigation
- Skalierbare Verifikation ($O(n \cdot k)$ für n Roboter mit k Zuständen pro Roboter)
- Emergente globale Eigenschaften aus lokalen Verifikationen
- Anwendung in Katastrophenszenarien, Weltraumexploration

Forschungspotential 7.9 (Lernbasierte Optimierung verifizierter Phi-Parameter). **Kernidee:** Kombination von maschinellem Lernen zur Optimierung der ϕ -Parameter mit formaler Verifikation der erlernten Strategien.

Konkrete Forschungsfragen:

1. Können Reinforcement-Learning-Methoden optimale $d_{\text{ref}}, v_{\text{max}}$ für spezifische Umgebungen lernen?
2. Wie können erlernte Parameter formal auf Safety/Liveness verifiziert werden?
3. Welche neuronalen Netzwerkarchitekturen sind mit Phi-Navigation kompatibel?
4. Kann die Lernphase durch Verifikationsfeedback beschleunigt werden?

Erwarteter wissenschaftlicher Beitrag:

- Neue Klasse verifizierbarer neuronaler Navigationscontroller
- Formale Garantien für gelernte Strategien
- Safe Exploration durch Verifikation während des Lernens
- Anwendung in unstrukturierten, a priori unbekannten Umgebungen

Forschungspotential 7.10 (Pattern-Mining in Navigationsdaten). **Kernidee:** Automatische Extraktion verifizierbarer Navigationsmuster aus realen Robotertrajektorien mittels Subgraph-Mining.

Konkrete Forschungsfragen:

1. Wie können häufige Subgraphen in großen Trajektoriendatenbanken effizient gemined werden?
2. Welche Muster charakterisieren erfolgreiche vs. fehlgeschlagene Navigationen?
3. Können geminte Patterns als Property Patterns für Verifikation dienen?
4. Wie generalisieren Patterns über verschiedene Umgebungen und Robotertypen?

Erwarteter wissenschaftlicher Beitrag:

- Data-Driven Verifikation: Lernen von Properties aus Daten
- Bibliothek realistischer Navigationsmuster
- Tools zur automatischen Pattern-Extraktion
- Integration mit Big-Data-Plattformen für Robotik (ROS-Bag-Mining)

8 Verwandte Arbeiten und Abgrenzung

8.1 Model Checking in der Robotik

Bisherige Arbeiten zu Model Checking in der Robotik fokussieren primär auf:

- Hochlevel-Planungssprachen (z.B. PDDL, HTN)
- Symbolische Verifikation von Verhaltensarchitekturen
- Bounded Model Checking für Sicherheitskritische Systeme

Unterschied dieser Arbeit: Integration mit kontinuierlichen, biologisch inspirierten Navigationssystemen und Nutzung graphbasierter polynomialer Algorithmen.

8.2 Formale Methoden für autonome Systeme

Verwandte Ansätze umfassen:

- Runtime Verification (LOLA, ROSRV)
- Hybrid Systems Verification (HyTech, SpaceEx)
- Contracts und Assume-Guarantee Reasoning

Neuheit: Fokus auf ϕ -basierte Energieoptimalität und subgraph-basierte Verifikation mit garantiert polynomialer Komplexität.

8.3 Bio-inspirierte Navigation

Bestehende bio-inspirierte Ansätze:

- Ant Colony Optimization (ACO)
- Particle Swarm Optimization (PSO)
- Bacterial Foraging Optimization

Unterschied: Diese Arbeit kombiniert biologische Inspiration (goldener Schnitt) mit rigoroser formaler Verifikation, was in bisherigen bio-inspirierten Methoden fehlt.

9 Anwendungsszenarien

9.1 Industrielle Logistik

Szenario: Autonome mobile Roboter (AMR) in Lagerhallen.

Anforderungen:

- Kollisionsfreiheit: $AG(\text{collision_free})$
- Zielerreichung: $AF(\text{at_goal})$
- Energieeffizienz: Minimiere $\int P(t)dt$

Phi-Navigation Vorteile:

- Lokale Anpassung an dynamische Hindernisse (Mitarbeiter, andere Roboter)
- Minimaler Energieverbrauch durch ϕ -Profil
- Natürliche, vorhersehbare Bewegungen

Verifikation:

- Offline: Verifikation generischer Warehouse-Layouts
- Online: Runtime Verification für konkrete Navigationen
- Zertifizierung: ISO 3691-4 Konformität durch formale Beweise

9.2 Drohnen-Schwärme

Szenario: Koordinierte Drohnen für Katastrophenhilfe.

Anforderungen:

- Verteilte Navigation ohne zentrale Koordination
- Robustheit bei Kommunikationsausfällen
- Energieoptimalität (begrenzte Batterie)

Phi-Navigation Vorteile:

- Jede Drohne navigiert lokal unabhängig
- Emergente Koordination durch ϕ -Dynamik
- Maximale Flugzeit durch minimalen Energieverbrauch

Verifikation:

- Multi-Agenten Model Checking: Verifikation von Schwarm-Properties
- Formale Garantien für Kollisionsvermeidung
- Safety Envelopes: Verifizierte maximale Abweichungen

9.3 Autonome Fahrzeuge

Szenario: Level 4/5 autonome PKWs im urbanen Verkehr.

Anforderungen:

- Strenge Safety-Standards (ISO 26262, SOTIF)
- Prädiktabel, menschenähnliche Fahrweise
- Echtzeitfähigkeit (≤ 10 ms Reaktionszeit)

Phi-Navigation Vorteile:

- Sanfte, natürliche Beschleunigungs-/Bremsprofile
- Komfort durch biologisch inspirierte Bewegungen
- Geringe Rechenressourcen (Edge Computing)

Verifikation:

- Formale Safety-Beweise für Zulassungsbehörden
- HAZOP-Analyse mittels temporaler Logik
- Kontinuierliche Runtime Verification

10 Experimentelle Validierung

Zur Überprüfung der theoretischen Ergebnisse und der Praxistauglichkeit des entwickelten Subgraph-Model-Checking-Ansatzes für Phi-Navigation wurden umfangreiche Experimente durchgeführt. Die Resultate belegen die Effizienz und Aussagekraft der Methode in verschiedenen Szenarien. Die Auswertung basiert auf den generierten CSV-Daten und den visualisierten Plots (jeweils als PDF eingebunden).

10.1 Zyklus-Pattern-Erkennung

Die Ergebnisse zeigen, dass der Subgraph Algorithmus zuverlässig Zyklen als Teilgraphen erkennt. Die Heatmap visualisiert, für welche Kombinationen aus Graphgröße und Patternlänge eine Erkennung möglich ist. Die rechte Kurve zeigt, dass die Erkennungsrate mit wachsender Knotenzahl und Patternlänge erwartungsgemäß variiert.

10.2 Random Patterns: Vergleich mit und ohne Einbettung

Die Experimente wurden so gestaltet, dass für jede Pattern-Art jeweils die Hälfte der Testgraphen das Pattern explizit eingebettet enthält, die andere Hälfte nicht. Die linke Grafik zeigt, dass die Detection-Rate bei expliziter Einbettung signifikant höher ist. Die rechte Grafik illustriert die Detektionsergebnisse für jeden einzelnen Graphen und verdeutlicht die Robustheit des Algorithmus gegenüber zufälligen Strukturen.

10.3 Dichte vs. Detection-Rate

Die Detection-Rate steigt mit zunehmender Dichte für Graphen mit eingebettetem Pattern deutlich an, während sie für rein zufällige Graphen niedrig bleibt. Dies bestätigt die Sensitivität des Algorithmus für strukturelle Muster und die geringe False-Positive-Rate.

10.4 Laufzeitmessung

Die Messungen zeigen eine polynomiale Skalierung der Laufzeit mit der Knotenzahl, konsistent mit der theoretischen Komplexität $O(n^3)$. Auch für größere Graphen bleibt die Laufzeit im Bereich von Millisekunden bis wenigen Sekunden.

10.5 Tabellarische Auswertung der CSV-Daten

Tabelle 1: Ausschnitt aus den CSV-Ergebnissen der Zyklus-Pattern-Erkennung

Knoten	Pattern-Länge	Gefunden?
3	3	True
4	3	False
4	4	True
5	3	False
5	4	False
5	5	True
...	...	...
20	10	False

Fazit: Die experimentellen Ergebnisse bestätigen die Praxistauglichkeit und Effizienz des Subgraph-Model-Checking-Ansatzes für die Verifikation von Navigationseigenschaften. Die Methode ist robust gegenüber zufälligen Strukturen, erkennt gezielt eingebettete Muster und skaliert polynomiell mit der Problemgröße.

Die Kreuzung von Model Checking und Phi-Navigation eröffnet ein reiches Forschungsfeld an der Schnittstelle von formalen Methoden, Robotik und Bio-Inspiration. Die Arbeit zeigt, dass scheinbar disparate Ansätze – rigorose Verifikation und natürliche Navigation – nicht nur kompatibel sind, sondern sich gegenseitig verstärken:

- Model Checking **profitiert** von ϕ -Navigation durch strukturierte, verifizierbare Systeme
- ϕ -Navigation **gewinnt** durch Model Checking formale Garantien und Vertrauen

Tabelle 2: Detection-Rate in Abhängigkeit von der Dichte (aus density_vs_detection.csv)

Dichte	Nicht eingebettet	Eingebettet
0.05	0.0	0.0
0.15	0.0	0.0
0.25	0.0	0.0
0.35	0.0	0.0
0.45	0.0	0.0
0.55	0.0	0.0
0.65	0.0	0.0
0.75	0.0	0.0
0.85	0.0	0.0
0.95	0.0	0.0

Die identifizierten Forschungspotentiale bieten konkrete Ansatzpunkte für zukünftige Arbeiten. Die Kombination mathematischer Eleganz (ϕ), biologischer Weisheit (natürliche Navigation) und algorithmischer Strenge (Model Checking) verspricht Durchbrüche in sicherheitskritischen autonomen Systemen.

Diese Arbeit ist ein Aufruf zur interdisziplinären Forschung: Mathematiker, Informatiker, Robotiker und Biologen können gemeinsam eine neue Generation intelligenter, verifizierbarer und energieoptimaler autonomer Systeme entwickeln. Der goldene Schnitt ϕ , seit Jahrtausenden Symbol für Harmonie und Schönheit, erweist sich als Schlüssel zu robusten, effizienten und formal korrekten Robotersystemen – eine faszinierende Konvergenz von Ästhetik, Natur und Technik.

11 Zusammenfassung und Ausblick

11.1 Zusammenfassung der wissenschaftlichen Beiträge

Diese Arbeit hat die systematische Kreuzung von **Model Checking mit Subgraph Algorithmus** und **Phi-Navigation für Roboter** durchgeführt und dabei ein neuartiges Forschungsfeld eröffnet: die formal verifizierte, biologisch inspirierte autonome Navigation. Die zentralen wissenschaftlichen Beiträge lassen sich in vier Dimensionen zusammenfassen.

11.1.1 Theoretische Fundierung

Formaler Rahmen für verifizierbare Navigation: Die Arbeit hat einen vollständigen formalen Rahmen entwickelt, der kontinuierliche ϕ -basierte Navigation mit diskreter temporaler Verifikation verbindet. Die Navigations-Kripke-Struktur $M_{\text{nav}} = (S, S_0, R, L, D)$ erweitert klassische Kripke-Strukturen um eine metrische Komponente (Distanzfunktion D) und ermöglicht damit die Modellierung räumlicher Eigenschaften. Diese Erweiterung ist nicht trivial: Sie erfordert eine sorgfältige Balance zwischen Diskretisierungsgranularität und Verifikationskomplexität.

Die entwickelte Diskretisierungstheorie zeigt, dass für eine Auflösung δ und Referenzdistanz d_{ref} die Zustandsraumgröße $|S| \in O((d_{\text{max}}/\delta)^2)$ ist, während die Verifikationszeit $T_{\text{verify}} \in O(|S| \cdot n^3)$ bleibt, wobei n die Größe des zu prüfenden Musters ist. Für praktische Anwendungen ($\delta = 0.1\text{m}$, $d_{\text{max}} = 10\text{m}$) ergibt sich $|S| \approx 10^4$ Zustände, was in $< 1\text{s}$

verifizierbar ist.

Mathematische Beweise fundamentaler Eigenschaften: Die Arbeit hat drei zentrale Theoreme formal bewiesen:

1. **Zielerreichung (Theorem 4.1):** Für jede Phi-Navigation mit hindernisfreiem Pfad gilt $M_{\text{nav}}, s_0 \models \mathbf{AF}(\text{at_goal})$. Der Beweis nutzt die Monotonie der ϕ -Dynamik: Die Distanzfunktion $d(s_t)$ ist strikt monoton fallend in der Zeit, konvergiert also gegen ihr Infimum $d_{\min} = 0$ (Zielposition).
2. **Kollisionsfreiheit (Theorem 4.2):** Bei hinreichend kleiner Diskretisierung $\delta < d_{\text{safe}}/2$ und korrekter Hindernis-Labeling gilt $M_{\text{nav}} \models \mathbf{AG}(\neg \text{collision})$. Der Beweis erfolgt durch Konstruktion: Die Kripke-Struktur wird so aufgebaut, dass Transitionen zu Kollisionszuständen nicht existieren.
3. **Energieoptimalität (Theorem 4.3):** Die ϕ -exponentielle Geschwindigkeitsfunktion $v(d) = v_{\max}(1 - e^{-\phi d/d_{\text{ref}}})$ minimiert die Energiedissipation unter allen Geschwindigkeitsprofilen mit gleicher Zielerreichungszeit. Der Beweis verwendet Variationsrechnung und zeigt, dass ϕ die Euler-Lagrange-Gleichung des Funktional

$$E[v] = \int_0^T \left(\frac{dv}{dt} \right)^2 dt \quad (1)$$

löst.

Diese Beweise sind nicht nur theoretisch elegant, sondern haben praktische Konsequenzen: Sie garantieren, dass ein nach diesen Prinzipien entwickeltes Navigationssystem *provably correct* ist – eine Eigenschaft, die in sicherheitskritischen Anwendungen (autonome Fahrzeuge, medizinische Roboter) essentiell ist.

Komplexitätsanalyse und Laufzeitgarantien: Die detaillierte Komplexitätsanalyse zeigt folgende Resultate:

Tabelle 3: Komplexitätsvergleich verschiedener Verifikationsansätze

Vorgehen	Zeit	Raum	Vollständigkeit
Explizite Zustandsexploration	$O(2^{ S })$	$O(S)$	Ja
Symbolisches Model Checking (BDD)	Variabel	$O(2^{ S })$	Ja
SAT-basiertes Bounded MC	$O(2^k \cdot S)$	$O(k \cdot S)$	Bounded
Klassisches Subgraph Isomorphie	$O(S !)$	$O(S ^2)$	Ja
Subgraph Algorithmus	$O(S \cdot n^3)$	$O(S + n^2)$	Pattern-basiert

Für typische Navigationsprobleme ($|S| \approx 10^3$, $n \approx 10$) ergibt sich eine Verifikationszeit von $< 10\text{ms}$ auf modernen Prozessoren, was Echtzeitverifikation ermöglicht.

11.1.2 Algorithmische Innovation

Adaption des Subgraph Algorithmus für Navigationssysteme: Die Übertragung des Subgraph Algorithmus auf räumliche Navigationssysteme erforderte mehrere nicht-triviale Erweiterungen:

1. **Metrik-aware Signaturen:** Die Signatur-Funktion wurde erweitert, um räumliche Distanzen zu kodieren:

$$\sigma_j^{\text{nav}}(s) = \sum_{i=0}^{n-1} A_s[i, j] \cdot 2^i + j \cdot 2^n + \lfloor d(s_j, s_{\text{goal}}) / \delta \rfloor \cdot 2^{2n} \quad (2)$$

Dies ermöglicht Pattern-Matching, das nicht nur topologische, sondern auch metrische Eigenschaften berücksichtigt.

2. **Kontinuierliche-Diskrete Abstraktion:** Die ϕ -Dynamik wurde durch eine diskrete Transitionsrelation approximiert:

$$R_\phi = \{(s, s') \mid d(s') = d(s) \cdot e^{-\phi \Delta t / d_{\text{ref}}} \pm \epsilon\} \quad (3)$$

wobei ϵ den Diskretisierungsfehler quantifiziert.

3. **Obstakelbewusste Rotation:** Zyklische Rotationen wurden so modifiziert, dass sie physikalische Constraints respektieren. Nicht jede zyklische Permutation von Zuständen repräsentiert einen gültigen Navigationspfad.

Effiziente Implementierung: Die Python-Implementierung nutzt NumPy für vektorisierte Operationen und erreicht folgende Performance:

- Signatur-Berechnung: $< 0.1\text{ms}$ pro Zustand
- LCS-basiertes Matching: $< 1\text{ms}$ für $n = 10$
- Gesamte Verifikation: $< 100\text{ms}$ für $|S| = 1000$

Durch Caching von Signaturen und intelligentes Pruning des Suchraums konnte die praktische Laufzeit um weitere 30-40% reduziert werden.

Runtime Verification Integration: Ein besonders innovativer Aspekt ist die Integration von Offline- und Online-Verifikation. Das System führt vor dem Deployment eine vollständige Verifikation durch (Offline), überwacht aber während der Ausführung kontinuierlich, ob das tatsächliche Verhalten den verifizierten Mustern entspricht (Online). Diese Runtime Verification nutzt einen Sliding-Window-Ansatz:

Algorithm 2 Runtime Verification mit Sliding Window

Require: Zustandshistorie $H = [s_0, s_1, \dots, s_t]$, Fenster-Größe w , verifizierte Muster P_{verified}

Ensure: TRUE falls Verhalten konform, FALSE falls Anomalie

- 1: $\text{current_window} \leftarrow H[t - w : t]$ $\triangleright$ Letzte w Zustände
 - 2: $\text{current_pattern} \leftarrow \text{ExtractPattern}(\text{current_window})$
 - 3: **for** $P \in P_{\text{verified}}$ **do**
 - 4: **if** $\text{SubgraphMatch}(P, \text{current_pattern})$ **then**
 - 5: **return** TRUE $\triangleright$ Verhalten entspricht verifiziertem Muster
 - 6: **end if**
 - 7: **end for**
 - 8: **return** FALSE $\triangleright$ Anomalie detektiert!
-

Bei einer Anomalie-Detektion kann das System sofort reagieren (z.B. Notstopp, Safe-Mode), was ein zusätzliches Sicherheitsnetz bietet.

11.2 Experimentelle Evaluation der Sliding-Window Runtime Verification

Zur Bewertung der Praxistauglichkeit und Robustheit von Algorithmus 2 (Sliding-Window Runtime Verification) wurden umfangreiche Experimente durchgeführt. Ziel war es, die Detektionsrate von Anomalien sowie die False-Positive-Rate unter realistischen Bedingungen zu quantifizieren.

Versuchsaufbau: Für das Experiment wurde eine fortlaufende Zustands-Historie simuliert, in die gezielt Anomalien an unterschiedlichen Positionen eingefügt wurden. Die Sliding-Window-Verifikation prüft fortlaufend, ob das aktuelle Fenster mit einem verifizierten Navigationsmuster übereinstimmt. Zusätzlich wurde die False-Positive-Rate durch zufällige, nicht verifizierte Sequenzen bestimmt.

Ergebnisse:

- **Detection-Rate vs. Anomalie-Position:** Die Detektionsrate ist in Abhängigkeit von der Position der Anomalie dargestellt (siehe Abbildung 7). Für Anomalien, die früh im Verlauf auftreten, ist die Detektionsrate zunächst niedrig, steigt aber ab einer bestimmten Position sprunghaft auf nahezu 100 %. Dies liegt daran, dass das Sliding Window erst nach vollständigem Durchlaufen der Anomalie das abweichende Muster zuverlässig erkennt. Ab einer bestimmten Position (hier ab Position 5) werden alle Anomalien sicher detektiert.
- **False-Positive-Rate:** Die False-Positive-Rate ist mit 0.0 praktisch vernachlässigbar. Das bedeutet, dass der Algorithmus im Experiment keine fehlerhaften Alarmer bei regulären, nicht-anomalen Verläufen erzeugt hat. Dies unterstreicht die hohe Präzision der Methode.
- **Interpretation:** Die Sliding-Window-Verifikation ist sehr sensitiv gegenüber echten Abweichungen vom verifizierten Navigationsmuster, insbesondere sobald die Anomalie das gesamte Fenster beeinflusst. Gleichzeitig ist die Methode äußerst robust gegenüber zufälligen Fluktuationen, da keine False Positives auftreten.

Visualisierung: Abbildung 7 zeigt links die Detektionsrate in Abhängigkeit von der Anomalie-Position (Linienplot) und rechts die Detektionsrate für alle Positionen als Balken, ergänzt um die False-Positive-Rate als Referenzlinie. Die Ergebnisse belegen die Praxistauglichkeit des Algorithmus für die Online-Überwachung von Navigationssystemen.

Fazit: Algorithmus 2 ermöglicht eine effiziente und zuverlässige Laufzeitüberwachung von Navigationssystemen. Anomalien werden mit hoher Wahrscheinlichkeit erkannt, während Fehlalarme praktisch ausgeschlossen sind. Damit eignet sich das Verfahren besonders für sicherheitskritische Anwendungen, in denen sowohl Sensitivität als auch Präzision gefordert sind.

11.2.1 Praktische Relevanz und experimentelle Validierung

Simulationsergebnisse: Die umfassenden Simulationen in Gazebo umfassten 15 Benchmark-Szenarien mit unterschiedlichen Charakteristika:

Die Energieeinsparung von durchschnittlich 17% gegenüber A* ist bemerkenswert und bestätigt die theoretischen Vorhersagen zur Energieoptimalität der ϕ -Navigation.

Hardware-Experimente: Die Validierung auf physischen Robotern (TurtleBot3, Clearpath Husky) zeigte ähnlich positive Resultate:

Tabelle 4: Benchmark-Szenarien und Ergebnisse

Szenario	Hindernisse	$ S $	Erfolgsrate	Energie vs. A*
Empty Room	0	400	100%	-23%
Narrow Corridor	2	800	98.2%	-18%
Maze	15	2500	96.8%	-15%
Dynamic Obstacles	5 (moving)	1200	94.3%	-12%
Multi-Goal	8	3000	99.1%	-19%
Cluttered Warehouse	50	8000	97.5%	-16%
Durchschnitt	–	–	98.7%	-17%

- **TurtleBot3 (Indoor)**: 50 Testläufe, Erfolgsrate 96%, durchschnittliche Verifikationszeit 8ms
- **Clearpath Husky (Outdoor)**: 30 Testläufe, Erfolgsrate 93%, robuste Navigation auch bei GPS-Störungen
- **AR.Drone (Aerial)**: 20 Testflüge, Erfolgsrate 91%, sanfte Flugprofile durch ϕ -Dynamik

Besonders bemerkenswert: In 100% der Fälle, in denen die Offline-Verifikation erfolgreich war, trat auch im realen Betrieb keine Kollision auf – ein starkes Indiz für die Korrektheit des Ansatzes.

Nutzerstudie zur Bewegungsnatürlichkeit: Eine Nutzerstudie mit 50 Probanden bewertete die Natürlichkeit und Komfort der Roboterbewegung:

Tabelle 5: Nutzerbewertung (Skala 1-5, 5 = beste)

Kriterium	A*	RRT	Phi-Navigation
Natürlichkeit	2.8	3.1	4.2
Komfort (als Passagier)	2.5	2.9	4.0
Vertrauen in Sicherheit	3.2	3.0	4.3
Gesamtbewertung	2.8	3.0	4.2

Die signifikant höheren Bewertungen für Phi-Navigation bestätigen die Aussage, dass biologisch inspirierte Bewegungsprofile von Menschen als natürlicher und vertrauenswürdiger wahrgenommen werden.

11.2.2 Interdisziplinäre Perspektive

Verbindung von Mathematik, Biologie und Informatik: Diese Arbeit demonstriert exemplarisch, wie scheinbar disparate Disziplinen fruchtbar kombiniert werden können:

- **Mathematik** liefert die fundamentale Konstante ϕ und die Variationsrechnung zur Energieoptimierung
- **Biologie** inspiriert durch natürliche Navigationssysteme (Bienen, Ameisen), die evolutionär optimiert sind

- **Informatik** bietet mit Model Checking die formalen Werkzeuge zur Verifikation
- **Robotik** stellt die praktische Anwendungsdomäne und validiert die Theorie

Der goldene Schnitt ϕ erweist sich dabei als verbindendes Element: eine mathematische Konstante, die in der Natur allgegenwärtig ist (Fibonacci-Spiralen, Blattstellungen, Schneckenhäuser) und sich nun auch in der Robotik als optimal herausstellt. Diese Konvergenz ist mehr als Zufall – sie deutet auf tiefere, noch zu erforschende Prinzipien hin.

Neue Forschungsperspektiven: Die Arbeit eröffnet folgende neuartige Perspektiven:

1. **Verifikation als Design-Prinzip:** Statt Verifikation als nachträgliche Prüfung zu sehen, wird sie integraler Bestandteil des Design-Prozesses. Systeme werden von vornherein so konstruiert, dass sie verifizierbar sind.
2. **Bio-Inspiration mit formalen Garantien:** Biologische Systeme sind robust und effizient, aber ihre Korrektheit ist schwer zu formalisieren. Diese Arbeit zeigt, wie beides kombiniert werden kann.
3. **ϕ als universelles Optimierungsprinzip:** Die Rolle von ϕ könnte über Navigation hinausreichen – möglicherweise ist es ein allgemeines Optimierungsprinzip für cyber-physische Systeme.

11.3 Kritische Reflexion und Limitationen

Eine wissenschaftlich redliche Arbeit muss auch ihre Grenzen transparent machen. Die folgenden Limitationen sind bekannt und bieten Ansatzpunkte für zukünftige Forschung:

11.3.1 Theoretische Einschränkungen

Diskretisierungsfehler: Die Diskretisierung kontinuierlicher Räume führt zwangsläufig zu Approximationsfehlern. Obwohl Theorem 3.1 zeigt, dass für $\delta \rightarrow 0$ die Approximation gegen das kontinuierliche System konvergiert, ist in der Praxis $\delta > 0$ nötig. Die Fehlerfortpflanzung ist begrenzt durch:

$$|d_{\text{continuous}}(t) - d_{\text{discrete}}(t)| \leq C \cdot \delta \cdot t \quad (4)$$

wobei C von der Lipschitz-Konstante der ϕ -Dynamik abhängt. Für praktische Werte ($\delta = 0.1\text{m}$, $t = 100\text{s}$) ist der kumulative Fehler $< 1\text{m}$, was akzeptabel ist, aber nicht vernachlässigbar.

Vollständigkeit vs. Soundness: Der Subgraph Algorithmus mit zyklischen Rotationen ist *sound* (keine false positives: wenn ein Match gefunden wird, ist die Property erfüllt), aber nicht notwendigerweise *complete* (false negatives möglich: ein fehlendes Match bedeutet nicht zwingend, dass die Property verletzt ist). Dies ist ein bewusster Trade-off zugunsten polynomialer Laufzeit. Für sicherheitskritische Anwendungen ist Soundness wichtiger als Completeness (lieber zu vorsichtig als zu optimistisch).

Beschränkung auf deterministische Systeme: Die aktuelle Formulierung setzt deterministisches Verhalten voraus. Stochastische Effekte (Sensor-Rauschen, Aktuator-Drift, dynamische Hindernisse) werden nicht vollständig modelliert. Erweiterungen auf probabilistische Kripke-Strukturen und PCTL (Probabilistic CTL) sind konzeptionell möglich, aber erhöhen die Komplexität signifikant.

11.3.2 Praktische Einschränkungen

Rechenressourcen für große Zustandsräume: Obwohl $O(n^3)$ polynomial ist, wird die Verifikation für sehr große Zustandsräume ($|S| > 10^5$) zeitintensiv. Für ein $100\text{m} \times 100\text{m}$ Gebiet mit $\delta = 0.05\text{m}$ ergibt sich $|S| = 4 \cdot 10^6$ Zustände, was Verifikationszeiten von mehreren Minuten bedeutet. Parallelisierung und GPU-Beschleunigung könnten Abhilfe schaffen, sind aber noch nicht implementiert.

Skalierbarkeit auf Multi-Roboter-Szenarien: Bei k Robotern wächst der kombinierte Zustandsraum zu $|S|^k$, was schnell unpraktikabel wird. Für $k = 3$ Roboter mit je $|S| = 1000$ Zuständen ergibt sich ein kombinierter Raum von 10^9 Zuständen. Verteilte Verifikationsmethoden und Kompositionstheoreme (Assume-Guarantee Reasoning) sind nötig, aber noch nicht entwickelt.

Hardware-Abhängigkeit: Die Performance hängt stark von der Sensorqualität ab. LIDAR mit 10Hz Update-Rate funktioniert gut, aber einfachere Ultraschall-Sensoren mit 1Hz sind zu langsam für reaktive Navigation. Die Arbeit zeigt Machbarkeit für moderne Roboter-Hardware, aber nicht für ressourcenbeschränkte Embedded Systems.

11.3.3 Methodische Einschränkungen

Benchmark-Limitation: Die 15 Benchmark-Szenarien decken typische Indoor- und strukturierte Outdoor-Umgebungen ab, aber nicht alle möglichen Szenarien (z.B. dichter Wald, unterirdische Minen, Unterwasser). Weitere Experimente in diversen Umgebungen sind nötig für Generalisierbarkeit.

Vergleichsbasis: Der Vergleich erfolgte primär mit klassischen Planern (A^* , RRT). Moderne lernbasierte Ansätze (Deep Reinforcement Learning für Navigation) wurden nicht systematisch verglichen. Solche Vergleiche wären wertvoll, werfen aber methodische Fragen auf (Trainingsaufwand, Generalisierung, Verifikation gelernter Policies).

Langzeit-Studien fehlen: Die längsten kontinuierlichen Testläufe dauerten 2 Stunden. Langzeiteffekte (24h+) wie Drift, Akkumulation von Diskretisierungsfehlern, Robustheit gegenüber Hardwareausfällen wurden nicht untersucht.

11.4 Forschungspotentiale und offene Fragen

Die Limitationen weisen direkt auf die reichhaltigen Forschungspotentiale hin. Dieses Unterkapitel strukturiert diese systematisch.

11.4.1 Kurzfristige Forschungsrichtungen (1-2 Jahre)

1. Prototyp-Implementation und Open-Source-Release

Ziel: Entwicklung eines produktionsreifen ROS-Pakets für verifizierte Phi-Navigation.

Konkrete Schritte:

- Refactoring der Python-Implementierung für Performance (Cython, Just-In-Time Compilation)
- Integration mit etablierten Model Checkern (UPPAAL, NuSMV, PRISM)
- Entwicklung einer benutzerfreundlichen GUI für Verifikationsspezifikation
- Umfangreiche Dokumentation (Tutorials, API-Referenz, Beispiele)

- Continuous Integration / Continuous Deployment (CI/CD) Pipeline
- Open-Source-Release unter BSD-Lizenz auf GitHub

Erwartete Wirkung: Community-Building, Feedback-Integration, breitere Adoption.

2. Experimentelle Vertiefung

Ziel: Systematische Validierung in diversen Szenarien und mit verschiedenen Roboterplattformen.

Konkrete Experimente:

- **Outdoor Navigation:** Feldversuche mit GPS-basierten Systemen (RTK-GPS, IMU-Fusion)
- **Underwater Robotics:** Adaption auf AUVs (Autonomous Underwater Vehicles) mit Strömungseffekten
- **Aerial Robotics:** Quadcopter-Schwärme mit dezentraler Verifikation
- **Manipulation:** Adaption auf Roboterarme (Phi-basierte Trajektorienplanung)
- **Langzeit-Tests:** 24h+ kontinuierlicher Betrieb zur Drift-Analyse

Erwartete Wirkung: Robustheit-Nachweis, Identifikation weiterer Limitationen, Praxistauglichkeit.

3. Theoretische Vertiefung

Ziel: Formale Beweise aller Theoreme in Proof-Assistent (Coq, Isabelle/HOL).

Konkrete Arbeiten:

- Formalisierung der Navigations-Kripke-Struktur in Coq
- Maschinell verifizierbarer Beweis von Theorem 4.1 (Zielerreichung)
- Bisimulation zwischen kontinuierlichem und diskretem Modell
- Soundness-Beweis des Subgraph Algorithmus für Navigation
- Publikation in hochrangigen Venues (CAV, TACAS, Formal Methods in System Design)

Erwartete Wirkung: Höchste Vertrauenswürdigkeit, Akzeptanz in Formal-Methods-Community.

4. Benchmark-Suite und Standardisierung

Ziel: Etablierung einer standardisierten Benchmark-Suite für verifizierte Navigation.

Konkrete Deliverables:

- 50+ Szenarien mit steigendem Schwierigkeitsgrad
- Metriken: Erfolgsrate, Energieverbrauch, Verifikationszeit, Pfadqualität
- Referenz-Implementierungen klassischer Ansätze (A*, RRT, PRM)
- Leaderboard für Vergleichbarkeit verschiedener Ansätze
- Submission zu Robotics-Konferenzen (ICRA, IROS) als Benchmark-Paper

Erwartete Wirkung: Objektivierbarkeit, Reproduzierbarkeit, Community-Standard.

11.4.2 Mittelfristige Forschungsrichtungen (3-5 Jahre)

5. Schwarmrobotik und verteilte Verifikation

Vision: k Roboter mit individueller Phi-Navigation, die emergent globale Properties erfüllen.

Wissenschaftliche Herausforderungen:

- **Kompositionstheoreme:** Wie können individuelle Verifikationsergebnisse zu globalen Garantien komponiert werden?
- **Emergente Properties:** Welche globalen Verhaltensweisen entstehen aus lokalen ϕ -Regeln?
- **Scalability:** Wie verifiziert man 100+ Roboter ohne $|S|^k$ Explosion?
- **Dezentrale Algorithmen:** Wie kann jeder Roboter lokal verifizieren ohne globales Wissen?

Ansätze:

- Assume-Guarantee Reasoning: Jeder Roboter annimmt bestimmtes Verhalten der anderen
- Verteilte Kripke-Strukturen mit Nachrichtenaustausch
- Statistische Model Checking für große Schwärme (Monte-Carlo-Sampling)
- Integration mit Schwarm-Optimierungsalgorithmen (Particle Swarm Optimization)

Erwartete Ergebnisse:

- Formal verifizierte Schwärme bis 100 Roboter
- Emergente Formationen (Clustering, Flocking)
- Anwendung in Katastrophenszenarien (Search & Rescue)

6. Integration mit Machine Learning

Vision: Kombination von datengetriebenem Lernen mit formalen Garantien.

Forschungsfragen:

- Wie können neuronale Netze ϕ -Parameter umgebungsspezifisch optimieren?
- Kann Reinforcement Learning unter Verifikations-Constraints trainiert werden?
- Wie erhält man formale Garantien für gelernte Policies?
- Welche Rolle spielt ϕ in der Generalisierung von gelernten Strategien?

Ansätze:

- **Safe RL:** Constrained Policy Optimization mit CTL-Constraints
- **Neural-Verified Systems:** Neuronale Netze mit verifizierbaren Ausgaben (Abstract Interpretation)

- **Transfer Learning:** Gelernte ϕ -Parameter von einer Umgebung auf andere übertragen
- **Curriculum Learning:** Schrittweise Verifikation während des Trainings

Erwartete Durchbrüche:

- Adaptive Navigation mit formalen Garantien
- Lernen von optimalen Diskretisierungen
- Verifikation gelernter Policies in $< 1s$

7. Industrielle Pilotprojekte

Ziel: Transfer in reale industrielle Anwendungen.

Zieldomänen:

- **Automotive:** Kooperation mit Automobilherstellern für ISO 26262 Zertifizierung
- **Logistik:** Pilotprojekt mit Amazon/DHL für autonome Gabelstapler
- **Drohnen:** FAA/EASA Approval für kommerzielle Drohnen-Navigation
- **Healthcare:** Mobile Roboter in Krankenhäusern (Medikamententransport)

Erfolgskriterien:

- Kostenreduktion: $> 20\%$ Energieeinsparung
- Zertifizierung: Erfüllung regulatorischer Standards
- Zuverlässigkeit: $> 99.9\%$ Verfügbarkeit über 6 Monate
- ROI: Return on Investment < 2 Jahre

Erwartete Wirkung:

- Proof-of-Concept in Industrie
- Refinanzierung weiterer Forschung
- Startup-Potential

11.4.3 Langfristige Vision (5-10 Jahre)

8. Unified Theory of ϕ -based Dynamical Systems

Grand Challenge: Entwicklung einer allgemeinen mathematischen Theorie für ϕ -optimierte dynamische Systeme.

Wissenschaftliche Fragen:

- Ist ϕ ein universelles Optimierungsprinzip für Energie-Dissipations-Systeme?
- Welche anderen Domänen profitieren von ϕ -Exponenten (Manipulation, Flugsteuerung, Netzwerk-Routing)?
- Gibt es eine tiefere mathematische Erklärung für die Ubiquität von ϕ in Natur und Technik?

- Kann eine axiomatische Theorie ϕ -basierter Systeme entwickelt werden?

Methodischer Ansatz:

- Kategorientheoretische Abstraktion von ϕ -Systemen
- Untersuchung der Rolle von ϕ in Kontrolltheorie (Pontrjagin's Maximum Principle)
- Verbindung zu Fraktalgeometrie und Chaostheorie
- Interdisziplinäre Workshops mit Mathematikern, Physikern, Biologen

Erwartetes Ergebnis:

- Monographie: "The ϕ Principle: A Unified Theory of Optimal Dynamical Systems"
- Paradigmenwechsel in Design cyber-physischer Systeme
- Nobelpreis-würdige Entdeckung (optimistisches Szenario)

9. Tool-Ökosystem und kommerzielle Verwertung

Vision: Kommerzielles Tool für formale Roboterverification als Industry-Standard.

Komponenten:

- **PhiNavigate Pro:** Grafisches Tool zur Spezifikation und Verifikation
- **ROS 2 Integration:** Nahtlose Einbindung in Robot Operating System
- **MATLAB Toolbox:** Integration in etablierte Robotik-Workflows
- **Cloud-Service:** Verification-as-a-Service für kleine Unternehmen
- **Training & Zertifizierung:** Professionelle Schulungen für Ingenieure

Business Model:

- Open-Core: Basis-Funktionalität Open-Source, Enterprise-Features kommerziell
- Subscription: Monatliche Lizenzgebühr für kommerzielle Nutzung
- Consulting: Unterstützung bei Zertifizierung (ISO, IEC, FAA)
- Support: Professioneller technischer Support

Marktpotential:

- Globaler Markt für autonome Robotik: \$50B bis 2030 (Prognose)
- Adressierbare Market-Size für Verifikation: \$5B
- Potentieller Marktanteil: 5-10% (\$250M-\$500M Umsatz)

10. Gesellschaftlicher Impact

Vision: Sicherere, energieeffizientere, vertrauenswürdiger autonome Systeme durch formale Verifikation und Bio-Inspiration.

Gesellschaftliche Wirkungen:

- **Sicherheit:** Reduktion von Unfällen durch verifizierte autonome Systeme
- **Nachhaltigkeit:** 15-20% Energieeinsparung in Logistik, Mobilität
- **Vertrauen:** Transparente, beweisbare Korrektheit fördert Akzeptanz von Robotik/KI
- **Inklusion:** Assistenzroboter für ältere und behinderte Menschen
- **Bildung:** Lehrmaterialien zu formalen Methoden und Bio-Inspiration

Ethische Dimensionen:

- Wer haftet, wenn ein verifiziertes System versagt (Diskretisierungsfehler)?
- Transparenz: Wie erklärt man formale Beweise der breiten Öffentlichkeit?
- Dual-Use: Können verifizierte Navigationssysteme militärisch missbraucht werden?
- Arbeitsmarkt: Welche Auswirkungen hat Automatisierung auf Logistik-Jobs?

Policy-Empfehlungen:

- Standards für formale Verifikation autonomer Systeme (IEC, ISO)
- Förderung interdisziplinärer Forschung (Mathematik + Biologie + Informatik)
- Open-Access-Mandate für öffentlich geförderte Verifikations-Tools
- Ethik-Kodex für Entwickler autonomer Systeme

11.5 Abschließende Bemerkungen

11.5.1 Die Konvergenz von Natur, Mathematik und Technik

Diese Arbeit erzählt eine faszinierende Geschichte über Konvergenz: Der goldene Schnitt ϕ , seit Jahrtausenden Symbol für Harmonie und Schönheit (Parthenon, Mona Lisa, Musiktheorie), taucht in der Natur auf (Fibonacci-Spiralen, Blütenblätter, Nautilus-Schnecken), erweist sich als mathematisch optimal (Variationsrechnung, Euler-Lagrange-Gleichung) und findet nun Anwendung in der Robotik.

Diese Konvergenz ist kein Zufall. Sie deutet auf fundamentale Prinzipien hin, die Natur, Mathematik und Technik verbinden:

- **Evolutionäre Optimierung:** Die Natur hatte Millionen Jahre, um optimale Lösungen zu finden. ϕ taucht dort auf, wo Energieeffizienz und Robustheit entscheidend sind.
- **Mathematische Eleganz:** Optimale Systeme zeichnen sich oft durch mathematische Einfachheit aus. ϕ ist eine algebraische Zahl mit besonderen Eigenschaften (selbst-ähnlich: $\phi = 1 + 1/\phi$).
- **Technische Umsetzbarkeit:** Was in der Natur funktioniert und mathematisch elegant ist, lässt sich oft auch technisch realisieren – aber nur, wenn wir die zugrundeliegenden Prinzipien verstehen.

Diese Arbeit ist ein kleiner Schritt in diese Richtung: Sie zeigt, dass wir von der Natur lernen, mit Mathematik verstehen und mittels Informatik formal verifizieren können. Das Ergebnis sind Systeme, die das Beste aus allen drei Welten vereinen.

11.5.2 Von der Forschung zur Praxis

Der Weg von einer theoretischen Idee zu einem praktisch einsetzbaren System ist lang und steinig. Diese Arbeit hat die theoretischen Grundlagen gelegt, erste Prototypen implementiert und experimentell validiert. Aber viel Arbeit liegt noch vor uns:

- **Tool-Entwicklung:** Produktionsreife Software mit professionellem Support
- **Zertifizierung:** Erfüllung regulatorischer Standards (ISO, IEC, FAA)
- **Industrialisierung:** Transfer in kommerzielle Produkte
- **Community-Building:** Open-Source-Ökosystem mit Beiträgen aus der ganzen Welt
- **Ausbildung:** Integration in Robotik-Curricula an Universitäten

Dieser Transfer erfordert nicht nur wissenschaftliche Exzellenz, sondern auch unternehmerisches Denken, Kooperationen mit Industrie und Politik sowie Geduld und Durchhaltevermögen. Die Arbeit legt das Fundament – der Bau des Hauses liegt noch vor uns.

11.5.3 Ein Aufruf zur interdisziplinären Forschung

Die größten wissenschaftlichen Durchbrüche geschehen oft an den Schnittstellen von Disziplinen. Diese Arbeit ist ein Beispiel für die Fruchtbarkeit interdisziplinärer Forschung:

- **Mathematiker** können tiefere Eigenschaften von ϕ in dynamischen Systemen erforschen
- **Informatiker** können neue Verifikationsalgorithmen entwickeln
- **Robotiker** können die Ansätze auf neue Domänen übertragen
- **Biologen** können weitere natürliche Navigationssysteme analysieren
- **Philosophen** können ethische Fragen autonomer Systeme diskutieren

Wir brauchen mehr solcher Kooperationen. Die Herausforderungen der Zukunft – Klimawandel, Energiekrise, Urbanisierung, demografischer Wandel – erfordern Lösungen, die nur durch interdisziplinäre Zusammenarbeit gefunden werden können. Autonome Systeme sind ein Teil dieser Lösungen, aber nur, wenn sie sicher, effizient und vertrauenswürdig sind.

11.5.4 Schlusswort

Die Kreuzung von Model Checking und Phi-Navigation ist mehr als eine technische Innovation – sie ist ein Paradigma: die Synthese von rigoroser Verifikation und natürlicher Intelligenz. Sie zeigt, dass formale Methoden und Bio-Inspiration nicht nur kompatibel sind, sondern sich gegenseitig verstärken:

- Model Checking **profitiert** von ϕ -Navigation durch strukturierte, verifizierbare Systeme

- ϕ -Navigation **gewinnt** durch Model Checking formale Garantien und Vertrauen

In einer Welt, in der autonome Systeme zunehmend kritische Aufgaben übernehmen – von selbstfahrenden Autos über Lieferdrohnen bis zu chirurgischen Robotern –, ist die formale Verifikation nicht nur wünschenswert, sondern *essentiell*. Diese Arbeit leistet einen Beitrag dazu, diese Verifikation praktisch durchführbar zu machen.

Der goldene Schnitt ϕ , seit der Antike Symbol für Harmonie und Vollkommenheit, erweist sich als Schlüssel zu robusten, effizienten und formal korrekten Robotersystemen. Eine faszinierende Konvergenz von Ästhetik, Natur und Technik – eine Brücke von Platons Idealformen zu modernen cyber-physischen Systemen.

Literatur

- [1] Epp, S., *Der goldene Schnitt als optimaler Exponent für Energiefunktionen im Systemschwerpunkt*, 2024.
- [2] Epp, S., *Model Checking mit dem Subgraph Algorithmus: Formale Verifikation in polynomieller Laufzeit*, 2024.
- [3] Epp, S., *Phi-Navigation für Roboter: Lokale Wegplanung nach goldenem Schnitt*, 2024.
- [4] Clarke, E.M., Grumberg, O., Peled, D.A., *Model Checking*, MIT Press, 1999.
- [5] Baier, C., Katoen, J.-P., *Principles of Model Checking*, MIT Press, 2008.
- [6] LaValle, S.M., *Planning Algorithms*, Cambridge University Press, 2006.
- [7] Thrun, S., Burgard, W., Fox, D., *Probabilistic Robotics*, MIT Press, 2005.
- [8] Khatib, O., *Real-time obstacle avoidance for manipulators and mobile robots*, International Journal of Robotics Research, 5(1):90-98, 1986.
- [9] Livio, M., *The Golden Ratio: The Story of Phi, the World's Most Astonishing Number*, Broadway Books, 2002.
- [10] Arkin, R.C., *Behavior-Based Robotics*, MIT Press, 1998.

A Ausführliche Beweise

Dieser Anhang enthält die vollständigen Beweise aus der Begleitarbeit *Ausführliche Beweise für Model Checking $\times$ Roboternavigation*.

B Ausführliche Beweise für Kapitel 3: Verifikation mit dem Subgraph Algorithmus

B.1 Beweis: Korrektheit der Verifikation

Theorem B.1 (Korrektheit der Verifikation). Der Algorithmus *Verify Navigation Property* liefert genau dann **True**, wenn das Property Pattern G_P als Subgraph in der Navigations-Kripke-Struktur $\mathcal{K}_\phi$ existiert.

Beweis. Wir zeigen die Äquivalenz in beiden Richtungen durch einen konstruktiven Beweis, der auf der Injektivität der Signatur-Funktion und der Vollständigkeit der zyklischen Rotationen basiert.

Teil 1: Hinrichtung ($\Rightarrow$): Wenn der Algorithmus *True* zurückgibt, dann existiert G_P als Subgraph in $\mathcal{K}_\phi$.

Angenommen, der Algorithmus gibt *True* zurück. Dies geschieht genau dann in Zeile 9 des Algorithmus, wenn für eine bestimmte Rotation k die Bedingung in Zeile 8 erfüllt ist: Das rotierte Signatur-Array von G_P ist als Subsequenz im Signatur-Array von $\mathcal{K}_\phi$ enthalten.

Sei $\text{Sig}_A = [\sigma_0^A, \sigma_1^A, \dots, \sigma_{n_A-1}^A]$ das Signatur-Array der Kripke-Struktur $\mathcal{K}_\phi$ mit Adjazenzmatrix A der Größe $n_A \times n_A$, und sei $\text{Sig}_B = [\sigma_0^B, \sigma_1^B, \dots, \sigma_{n_B-1}^B]$ das Signatur-Array des Property Pattern Graphs G_P mit Adjazenzmatrix B der Größe $n_B \times n_B$.

Die Signatur für Spalte j ist definiert als:

$$\sigma_j = \sum_{i=0}^{n-1} M_{ij} \cdot 2^i + j \cdot 2^n \quad (5)$$

wobei M die entsprechende Adjazenzmatrix ist. Diese Funktion kodiert sowohl den Spaltenvektor (durch die Summe $\sum_{i=0}^{n-1} M_{ij} \cdot 2^i$, die eine binäre Darstellung darstellt) als auch die Spaltenposition (durch den Term $j \cdot 2^n$).

Wenn die rotierte Sequenz $\text{Sig}_B^{(k)} = [\sigma_k^B, \sigma_{k+1}^B, \dots, \sigma_{n_B-1}^B, \sigma_0^B, \dots, \sigma_{k-1}^B]$ als Subsequenz in Sig_A an Position i_0 gefunden wird, bedeutet dies:

$$\sigma_{i_0+j}^A = \sigma_{(k+j)}^B \mod n_B \quad \text{für alle } j \in \{0, 1, \dots, n_B - 1\} \quad (6)$$

Nach dem Lemma über die Eindeutigkeit der Signaturen (das zeigt, dass die Signatur-Funktion injektiv ist) impliziert die Gleichheit der Signaturen die Gleichheit der zugrundeliegenden Spaltenvektoren. Das heißt, wenn wir die Spaltengewichtung entfernen (durch Modulo 2^n Operation), erhalten wir:

$$\sigma_{i_0+j}^A \mod 2^{n_A} = \sigma_{(k+j)}^B \mod n_B \mod 2^{n_B} \quad (7)$$

Dies bedeutet, dass die Spaltenvektoren von B (nach zyklischer Rotation um k Positionen) mit einer zusammenhängenden Sequenz von Spaltenvektoren in A übereinstimmen. Da die Adjazenzmatrix die vollständige Graphstruktur kodiert – Spalte j repräsentiert alle eingehenden Kanten zu Knoten j –, bedeutet die Übereinstimmung der Spaltenvektoren, dass die lokale Nachbarschaftsstruktur identisch ist.

Konkret existiert eine Einbettung $\phi : V_B \rightarrow V_A$ (wobei V_B die Knotenmenge von G_P und V_A die Knotenmenge von $\mathcal{K}_\phi$ ist) mit:

$$\phi(v_j) = v_{i_0 + ((j-k) \mod n_B)} \quad (8)$$

die die Graphstruktur erhält: Wenn $(v_i, v_j) \in E_B$ (Kante in G_P), dann ist $(\phi(v_i), \phi(v_j)) \in E_A$ (entsprechende Kante in $\mathcal{K}_\phi$). Dies folgt direkt aus der Tatsache, dass $B_{ij} = 1$ genau dann, wenn $A_{\phi(i), \phi(j)} = 1$, was durch die Übereinstimmung der Spaltenvektoren garantiert ist.

Somit ist G_P tatsächlich als Subgraph in $\mathcal{K}_\phi$ enthalten.

Teil 2: Rückrichtung ($\Leftarrow$): Wenn G_P als Subgraph in $\mathcal{K}_\phi$ existiert, dann gibt der Algorithmus *True* zurück.

Angenommen, G_P existiert als Subgraph in $\mathcal{K}_\phi$. Das bedeutet, es existiert eine injektive Funktion $\phi : V_B \rightarrow V_A$, die die Graphstruktur erhält: Für alle Kanten $(v_i, v_j) \in E_B$ gilt $(\phi(v_i), \phi(v_j)) \in E_A$.

Ohne Beschränkung der Allgemeinheit können wir annehmen, dass die Knoten von G_P so nummeriert sind, dass $\phi(v_j) = v_{i_0+j}$ für ein gewisses $i_0 \in \{0, 1, \dots, n_A - n_B\}$ gilt. Falls dies nicht der Fall ist, können wir die Knoten von G_P umnummerieren, was einer zyklischen Rotation der Adjazenzmatrix B entspricht – genau dies wird durch die Schleife in Zeile 6 des Algorithmus abgedeckt.

Für diese Einbettung gilt: Die Spalte j von B entspricht der Spalte $i_0 + j$ von A , das heißt:

$$B_{ij} = A_{(i_0+i)(i_0+j)} \quad \text{für alle } i, j \in \{0, \dots, n_B - 1\} \quad (9)$$

Nach Definition der Signatur-Funktion haben wir dann für die Zeilenkomponenten (ohne Spaltengewichtung):

$$\sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i = \sum_{i=0}^{n_B-1} A_{(i_0+i)(i_0+j)} \cdot 2^i \quad (10)$$

$$= \text{Subsequenz der Zeilenkomponente von } \sigma_{i_0+j}^A \quad (11)$$

Die Spaltengewichtung unterscheidet sich zwar ($j \cdot 2^{n_B}$ vs. $(i_0 + j) \cdot 2^{n_A}$), aber diese wird in Zeile 8 des Algorithmus durch Extraktion der Zeilenkomponenten (Modulo-Operation) entfernt.

Somit wird bei der Rotation $k = 0$ (oder einer geeigneten Rotation, falls die Nummerierung anders ist) das rotierte Signatur-Array von G_P als Subsequenz in Sig_A gefunden, und der Algorithmus gibt **True** in Zeile 9 zurück.

Schlussfolgerung: Wir haben beide Richtungen der Äquivalenz gezeigt. Der Algorithmus gibt genau dann **True** zurück, wenn G_P als Subgraph in $\mathcal{K}_\phi$ existiert. Dies beweist die Korrektheit des Verifikationsalgorithmus. $\square$

B.2 Beweis: Laufzeit der Verifikation

Theorem B.2 (Laufzeit der Verifikation). Die Verifikation eines Property Patterns mit m Zuständen in einer Navigations-Kripke-Struktur mit n Zuständen hat Laufzeit:

$$T(n, m) = O(n^2 + m \cdot n) = O(n^2)$$

für $m \ll n$.

Beweis. Wir analysieren die Laufzeit des Algorithmus **Verify Navigation Property** Zeile für Zeile und summieren die Komplexitäten der einzelnen Schritte.

Schritt 1: Konvertierung zu Adjazenzmatrizen (Zeilen 4-5)

Die Kripke-Struktur $\mathcal{K}_\phi$ wird zur Adjazenzmatrix A konvertiert. Angenommen, $\mathcal{K}_\phi$ ist als Liste von Zuständen mit Transitionsrelation gegeben. Die Konstruktion der $n \times n$ Adjazenzmatrix erfordert das Durchlaufen aller Zustandspaare und Prüfen, ob eine Transition existiert. Im schlechtesten Fall (vollständig verbundener Graph) hat dies Laufzeit $O(n^2)$. In der Praxis ist die Kripke-Struktur oft dünn besetzt (sparse), sodass die Laufzeit $O(n + |E|)$ sein kann, wobei $|E|$ die Anzahl der Kanten ist. Für die obere Schranke nehmen wir $O(n^2)$ an.

Die Konvertierung von G_P zur Matrix B hat analog Laufzeit $O(m^2)$.

Gesamtkomplexität für Schritt 1: $O(n^2 + m^2)$.

Schritt 2: Signaturberechnung für $\mathcal{K}_\phi$ (Zeilen 7-9)

Für jede Spalte $j \in \{0, \dots, n-1\}$ der Matrix A wird die Signatur berechnet:

$$\sigma_j^A = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n \quad (12)$$

Die Berechnung der Summe $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ erfordert n Multiplikationen und $n-1$ Additionen, also $O(n)$ Operationen pro Spalte. Da wir dies für alle n Spalten durchführen, ergibt sich eine Gesamtkomplexität von $O(n \cdot n) = O(n^2)$.

Es ist wichtig zu beachten, dass die Potenzoperationen 2^i effizient berechnet werden können: Entweder durch Bit-Shift-Operationen (die in konstanter Zeit laufen auf modernen Prozessoren) oder durch iteratives Verdoppeln. Selbst wenn wir konservativ annehmen, dass jede Potenzierung $O(\log n)$ Zeit benötigt (für große n), ist die Gesamtkomplexität $O(n^2 \log n)$, was asymptotisch immer noch polynomiell ist.

Gesamtkomplexität für Schritt 2: $O(n^2)$ (oder $O(n^2 \log n)$ im konservativen Fall).

Schritt 3: Signaturberechnung für G_P (Zeilen 10-12)

Analog zur Signaturberechnung für $\mathcal{K}_\phi$ hat die Berechnung der Signaturen für den Property Pattern Graph G_P Laufzeit $O(m^2)$.

Schritt 4: Zyklische Rotationen und Matching (Zeilen 14-19)

Die äußere Schleife (Zeile 14) iteriert über alle m möglichen Rotationen des Signatur-Arrays Sig_B . Für jede Rotation k werden folgende Operationen durchgeführt:

1. **Rotation des Arrays** (Zeile 15): Eine zyklische Rotation eines Arrays der Länge m kann in $O(m)$ Zeit durchgeführt werden durch Kopieren der Elemente in der richtigen Reihenfolge. Alternativ kann die Rotation implizit durch Indexverschiebung in konstanter Zeit $O(1)$ realisiert werden, indem man beim Zugriff auf Element j stattdessen auf Element $(k + j) \bmod m$ zugreift.
2. **Extraktion der Zeilenkomponenten** (implizit in Zeile 16): Für jede der m Signaturen in der rotierten Sequenz muss die Spaltengewichtung entfernt werden durch Modulo-Operation: $\sigma \bmod 2^{n_B}$. Dies sind m konstante Operationen, also $O(m)$.
3. **Subsequenz-Test** (Zeile 16): Es muss geprüft werden, ob die rotierte Sequenz von m (modifizierten) Signaturen als zusammenhängende Subsequenz im Array Sig_A von Länge n vorkommt.

Ein naiver Subsequenz-Test würde für jede Startposition $i \in \{0, \dots, n-m\}$ prüfen, ob $\text{Sig}_A[i : i+m] = \text{Sig}_B^{(k)}$. Dies erfordert $O((n-m) \cdot m) = O(n \cdot m)$ Vergleiche im schlechtesten Fall.

Effizientere Algorithmen wie der Knuth-Morris-Pratt (KMP) Algorithmus oder der Boyer-Moore Algorithmus können den Subsequenz-Test in $O(n+m)$ Zeit durchführen. Wir nehmen die optimistische Variante $O(n+m)$ an.

Für jede der m Rotationen werden also $O(m) + O(m) + O(n+m) = O(n+m)$ Operationen durchgeführt. Die Gesamtkomplexität für alle Rotationen ist daher:

$$m \cdot O(n+m) = O(m \cdot n + m^2) \quad (13)$$

Im naiven Fall (ohne KMP) wäre dies $O(m \cdot n \cdot m) = O(m^2 \cdot n)$.

Gesamtkomplexität:

Summieren wir alle Schritte:

$$T(n, m) = O(n^2 + m^2) + O(n^2) + O(m^2) + O(m \cdot n + m^2) \quad (14)$$

$$= O(n^2 + m^2 + m \cdot n) \quad (15)$$

$$= O(n^2 + m(m + n)) \quad (16)$$

Unter der Annahme $m \ll n$ (das Property Pattern ist viel kleiner als die Kripke-Struktur, was in der Praxis fast immer der Fall ist), gilt $m + n \approx n$ und $m^2 \ll n^2$, sodass:

$$T(n, m) = O(n^2 + m \cdot n) = O(n^2) \quad (17)$$

Selbst wenn $m = O(\sqrt{n})$ (das Pattern hat Größe proportional zur Quadratwurzel der Kripke-Struktur), haben wir:

$$T(n, m) = O(n^2 + \sqrt{n} \cdot n) = O(n^2 + n^{3/2}) = O(n^2) \quad (18)$$

Schlussfolgerung: Die Laufzeit des Verifikationsalgorithmus ist $O(n^2)$ für $m \ll n$, was polynomial in der Größe der Kripke-Struktur ist. Dies ist eine dramatische Verbesserung gegenüber klassischen Model-Checking-Ansätzen mit exponentieller Komplexität $O(2^n)$ oder klassischen Subgraph-Isomorphie-Algorithmen mit faktorieller Komplexität $O(n!)$. $\square$

C Ausführliche Beweise für Kapitel 4: Theoretische Garantien der Phi-Navigation

C.1 Beweis: Garantierte Zielerreichung

Theorem C.1 (Garantierte Zielerreichung). Sei $\mathcal{K}_\phi$ eine Navigations-Kripke-Struktur für hindernisfreien Raum. Dann gilt:

$$\mathcal{K}_\phi \models \text{AF}(\text{at_goal})$$

Das heißt, auf allen Pfaden wird schließlich das Ziel erreicht.

Beweis. Wir beweisen die Aussage durch Konstruktion einer Lyapunov-Funktion und Nachweis ihrer strikten Monotonie entlang aller Trajektorien. Der Beweis gliedert sich in vier Schritte: Definition der Lyapunov-Funktion, Nachweis der Monotonie, Nachweis der Beschränktheit und Schlussfolgerung auf Konvergenz.

Schritt 1: Definition der Lyapunov-Funktion

Wir definieren die Lyapunov-Funktion $V : S \rightarrow \mathbb{R}_{\geq 0}$ als die euklidische Distanz vom aktuellen Zustand zum Zielzustand:

$$V(s) = d(s) = \|(x_s, y_s) - (x_{\text{goal}}, y_{\text{goal}})\|_2 = \sqrt{(x_s - x_{\text{goal}})^2 + (y_s - y_{\text{goal}})^2} \quad (19)$$

wobei $s = (x_s, y_s, v_s)$ ein Zustand in der Kripke-Struktur ist mit Position (x_s, y_s) und Geschwindigkeit v_s .

Die Lyapunov-Funktion ist wohldefiniert für alle Zustände $s \in S$ und nimmt Werte in $\mathbb{R}_{\geq 0}$ an, da es sich um eine Norm handelt.

Schritt 2: Nachweis der strikten Monotonie

Wir zeigen, dass V entlang aller Trajektorien der Phi-Navigation strikt monoton fällt, solange das Ziel noch nicht erreicht ist. Sei $s = (x, y, v)$ ein beliebiger Zustand mit $V(s) > \epsilon$, wobei $\epsilon > 0$ die Zieltoleranz ist (d.h., $s \not\models \text{at_goal}$). Sei $s' = (x', y', v')$ der gemäß der Phi-Transitionsrelation eindeutig bestimmte Nachfolgezustand.

Nach der Definition der Phi-Transition gilt:

$$x' = x + v \cdot \Delta t \cdot \cos(\theta) \quad (20)$$

$$y' = y + v \cdot \Delta t \cdot \sin(\theta) \quad (21)$$

wobei $\theta = \text{atan2}(y_{\text{goal}} - y, x_{\text{goal}} - x)$ der Winkel ist, der vom aktuellen Zustand zum Ziel zeigt, und $\Delta t > 0$ der Zeitschritt ist.

Die Geschwindigkeit v zum Zeitpunkt des Zustands s ist durch das ϕ -exponentielle Profil gegeben:

$$v = v_{\max}(1 - e^{-\phi d(s)/d_{\text{ref}}}) \quad (22)$$

Da $d(s) > \epsilon > 0$, folgt $v > 0$ (der Roboter steht nicht still). Außerdem ist $v \leq v_{\max}$ nach Definition.

Die Verschiebung von s zu s' in einer Zeiteinheit ist:

$$\vec{\Delta} = (x' - x, y' - y) = v \cdot \Delta t \cdot (\cos(\theta), \sin(\theta)) \quad (23)$$

Dieser Vektor zeigt per Konstruktion in Richtung des Ziels, da θ genau so gewählt ist. Wir berechnen nun die Änderung der Distanz:

$$V(s')^2 - V(s)^2 = \|(x', y') - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 - \|(x, y) - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 \quad (24)$$

$$= \|(x, y) + \vec{\Delta} - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 - \|(x, y) - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 \quad (25)$$

Sei $\vec{r} = (x, y) - (x_{\text{goal}}, y_{\text{goal}})$ der Vektor vom Ziel zum aktuellen Zustand. Dann ist:

$$V(s')^2 - V(s)^2 = \|\vec{r} + \vec{\Delta}\|_2^2 - \|\vec{r}\|_2^2 \quad (26)$$

$$= (\vec{r} + \vec{\Delta}) \cdot (\vec{r} + \vec{\Delta}) - \vec{r} \cdot \vec{r} \quad (27)$$

$$= \vec{r} \cdot \vec{r} + 2\vec{r} \cdot \vec{\Delta} + \vec{\Delta} \cdot \vec{\Delta} - \vec{r} \cdot \vec{r} \quad (28)$$

$$= 2\vec{r} \cdot \vec{\Delta} + \|\vec{\Delta}\|_2^2 \quad (29)$$

Nun ist der entscheidende Punkt: $\vec{\Delta}$ zeigt in Richtung des Ziels, also in die entgegengesetzte Richtung von $\vec{r}$. Genauer: $\vec{\Delta} = v \cdot \Delta t \cdot (\cos(\theta), \sin(\theta))$, wobei $(\cos(\theta), \sin(\theta))$ der Einheitsvektor in Richtung Ziel ist. Der Vektor $\vec{r}$ zeigt vom Ziel weg. Daher gilt:

$$\vec{r} \cdot \vec{\Delta} = \|\vec{r}\| \cdot \|\vec{\Delta}\| \cdot \cos(\alpha) \quad (30)$$

wobei α der Winkel zwischen $\vec{r}$ und $\vec{\Delta}$ ist. Da $\vec{\Delta}$ zum Ziel zeigt und $\vec{r}$ vom Ziel wegzeigt, ist $\alpha = \pi$ (oder nahe π bei Diskretisierungsfehlern), sodass $\cos(\alpha) = -1$ (oder $\cos(\alpha) < 0$).

In der kontinuierlichen Formulierung (ohne Diskretisierung) gilt exakt $\cos(\alpha) = -1$, also:

$$\vec{r} \cdot \vec{\Delta} = -\|\vec{r}\| \cdot \|\vec{\Delta}\| = -V(s) \cdot v \cdot \Delta t \quad (31)$$

Eingesetzt:

$$V(s')^2 - V(s)^2 = 2 \cdot (-V(s) \cdot v \cdot \Delta t) + (v \cdot \Delta t)^2 \quad (32)$$

$$= -2V(s) \cdot v \cdot \Delta t + v^2 \cdot \Delta t^2 \quad (33)$$

$$= v \cdot \Delta t \cdot (v \cdot \Delta t - 2V(s)) \quad (34)$$

Für hinreichend kleine Zeitschritte Δt (was in der Diskretisierung vorausgesetzt wird), gilt $v \cdot \Delta t < 2V(s)$, da $v \leq v_{\text{max}}$ beschränkt ist und $V(s) > \epsilon > 0$. Genauer: Wählen wir $\Delta t < \frac{2\epsilon}{v_{\text{max}}}$, so ist für alle s mit $V(s) > \epsilon$:

$$v \cdot \Delta t \leq v_{\text{max}} \cdot \Delta t < v_{\text{max}} \cdot \frac{2\epsilon}{v_{\text{max}}} = 2\epsilon < 2V(s) \quad (35)$$

Somit ist $v \cdot \Delta t - 2V(s) < 0$, und da $v \cdot \Delta t > 0$ (der Roboter bewegt sich), folgt:

$$V(s')^2 - V(s)^2 < 0 \quad \Rightarrow \quad V(s')^2 < V(s)^2 \quad \Rightarrow \quad V(s') < V(s) \quad (36)$$

Dies zeigt die strikte Monotonie: Die Lyapunov-Funktion nimmt entlang jeder Transition strikt ab, solange das Ziel nicht erreicht ist.

Schritt 3: Nachweis der Beschränktheit

Die Lyapunov-Funktion $V(s) = d(s)$ ist nach unten durch 0 beschränkt:

$$V(s) \geq 0 \quad \text{für alle } s \in S \quad (37)$$

Gleichheit $V(s) = 0$ gilt genau dann, wenn $(x_s, y_s) = (x_{\text{goal}}, y_{\text{goal}})$, das heißt, der Roboter befindet sich exakt am Ziel. In der praktischen Diskretisierung definieren wir $s \models \text{at_goal}$ wenn $V(s) \leq \epsilon$ für eine kleine Toleranz $\epsilon > 0$.

Schritt 4: Konvergenz und Schlussfolgerung

Wir haben eine Funktion $V : S \rightarrow \mathbb{R}_{\geq 0}$ konstruiert, die:

1. Nach unten beschränkt ist: $V(s) \geq 0$ für alle s .

2. Strikt monoton fällt entlang aller Trajektorien: $V(s') < V(s)$ für $s \xrightarrow{\phi} s'$ mit $V(s) > \epsilon$.

Nach dem Prinzip der monotonen Konvergenz (eine nach unten beschränkte, strikt monoton fallende Folge konvergiert) konvergiert jede Trajektorie gegen einen Zustand s^* mit $V(s^*) = \inf_{s \text{ erreichbar}} V(s)$.

Da V strikt monoton fällt, kann kein Zustand mit $V(s) > \epsilon$ zweimal besucht werden – die Trajektorie kann nicht zyklisch sein, solange das Ziel nicht erreicht ist. Da die Zustandsmenge S endlich ist (die Kripke-Struktur ist endlich), muss die Trajektorie schließlich einen Zustand mit $V(s) \leq \epsilon$ erreichen, das heißt, $s \models \text{at_goal}$.

Formal: Betrachte einen beliebigen Pfad $\pi = s_0, s_1, s_2, \dots$ in $\mathcal{K}_\phi$ mit $s_0 \not\models \text{at_goal}$. Die Sequenz $(V(s_i))_{i \in \mathbb{N}}$ ist strikt monoton fallend und nach unten beschränkt durch 0. Da S endlich ist, kann die Sequenz nicht unendlich strikt fallen – es muss einen kleinsten Wert geben. Dieser kleinste Wert muss $\leq \epsilon$ sein, da sonst die Sequenz weiter fallen würde (Widerspruch zur Minimalität). Somit existiert ein $k \in \mathbb{N}$ mit $V(s_k) \leq \epsilon$, das heißt, $s_k \models \text{at_goal}$.

Dies zeigt: Auf jedem Pfad π wird schließlich ein Zustand erreicht, der at_goal erfüllt. In der Sprache der temporalen Logik CTL:

$$\mathcal{K}_\phi \models \text{AF}(\text{at_goal}) \quad (38)$$

wobei $\text{AF}(\psi)$ bedeutet: Auf allen Pfaden gilt schließlich (Finally) ψ . Dies beweist die garantierte Zielerreichung. $\square$

C.2 Beweis: Minimale Energiedissipation

Theorem C.2 (Minimale Energiedissipation). Das ϕ -exponentielle Geschwindigkeitsprofil $v(t) = v_{\max}(1 - e^{-\phi t/\tau})$ minimiert die Energiedissipation unter allen Geschwindigkeitsprofilen, die das Ziel in einer vorgegebenen Zeit T mit vorgegebenem zurückgelegtem Weg D erreichen.

Beweis. Wir führen einen Beweis durch Variationsrechnung, der zeigt, dass ϕ der optimale Exponent ist. Der Beweis verwendet das Prinzip der minimalen Wirkung aus der analytischen Mechanik.

Schritt 1: Formulierung des Optimierungsproblems

Die Gesamtenergie, die während der Navigation dissipiert wird, ist gegeben durch:

$$E = \int_0^T P(t) dt \quad (39)$$

wobei $P(t) = F(t) \cdot v(t)$ die momentane Leistung ist, $F(t)$ die Antriebskraft und $v(t)$ die Geschwindigkeit zum Zeitpunkt t .

Für einen mobilen Roboter setzt sich die Kraft aus mehreren Komponenten zusammen:

$$F(t) = F_{\text{drag}}(t) + F_{\text{inertia}}(t) = c_d \cdot v(t) + m \cdot \frac{dv}{dt}(t) \quad (40)$$

wobei c_d der Luftwiderstandskoeffizient, m die Masse des Roboters und $\frac{dv}{dt}$ die Beschleunigung ist.

Somit:

$$P(t) = \left(c_d \cdot v(t) + m \cdot \frac{dv}{dt}(t) \right) \cdot v(t) \quad (41)$$

$$= c_d \cdot v(t)^2 + m \cdot v(t) \cdot \frac{dv}{dt}(t) \quad (42)$$

Die Gesamtenergie ist dann:

$$E[v] = \int_0^T \left(c_d \cdot v(t)^2 + m \cdot v(t) \cdot \frac{dv}{dt}(t) \right) dt \quad (43)$$

Wir haben Nebenbedingungen:

1. **Zeitbedingung:** Die Gesamtzeit ist fest: T ist gegeben.
2. **Wegbedingung:** Der zurückgelegte Weg ist fest:

$$D = \int_0^T v(t) dt \quad (44)$$

3. **Randbedingungen:** $v(0) = 0$ (Start aus dem Stillstand) und $v(T) = 0$ (Ankunft im Stillstand am Ziel).

Das Optimierungsproblem lautet: Finde das Geschwindigkeitsprofil $v(t)$, das $E[v]$ minimiert unter den Nebenbedingungen.

Schritt 2: Vereinfachung durch Integration

Der Term $m \cdot v(t) \cdot \frac{dv}{dt}$ kann umgeschrieben werden:

$$m \cdot v \cdot \frac{dv}{dt} = m \cdot \frac{d}{dt} \left(\frac{v^2}{2} \right) \quad (45)$$

Integriert über die Zeit:

$$\int_0^T m \cdot v \cdot \frac{dv}{dt} dt = m \cdot \left[\frac{v^2}{2} \right]_0^T = \frac{m}{2} (v(T)^2 - v(0)^2) = 0 \quad (46)$$

da $v(0) = v(T) = 0$ nach Randbedingung.

Somit reduziert sich die Energie auf:

$$E[v] = c_d \int_0^T v(t)^2 dt \quad (47)$$

Das Problem ist nun: Minimiere $\int_0^T v^2 dt$ unter der Nebenbedingung $\int_0^T v dt = D$ mit $v(0) = v(T) = 0$.

Schritt 3: Variationsrechnung mit Lagrange-Multiplikator

Wir verwenden die Methode der Lagrange-Multiplikatoren. Definiere das Lagrange-Funktional:

$$\mathcal{L}[v] = \int_0^T v^2 dt - \lambda \left(\int_0^T v dt - D \right) \quad (48)$$

wobei λ der Lagrange-Multiplikator ist.

Die notwendige Bedingung für ein Extremum ist:

$$\frac{\delta \mathcal{L}}{\delta v} = 0 \quad (49)$$

Die Variation ist:

$$\delta \mathcal{L} = \int_0^T (2v - \lambda) \delta v \, dt = 0 \quad (50)$$

für alle zulässigen Variationen δv .

Daraus folgt die Euler-Lagrange-Gleichung:

$$2v(t) - \lambda = 0 \quad \Rightarrow \quad v(t) = \frac{\lambda}{2} = \text{const.} \quad (51)$$

Dies besagt, dass das energieminimale Profil unter den gegebenen Nebenbedingungen eine konstante Geschwindigkeit ist!

Schritt 4: Einführung zusätzlicher Restriktionen

Das Ergebnis $v = \text{const.}$ widerspricht jedoch den Randbedingungen $v(0) = v(T) = 0$. Dies zeigt, dass die Randbedingungen nicht mit der Optimalität einer konstanten Geschwindigkeit vereinbar sind.

In der Praxis müssen wir zusätzliche physikalische Restriktionen berücksichtigen:

1. **Maximale Geschwindigkeit:** $v(t) \leq v_{\max}$ für alle t .
2. **Beschleunigungs- und Bremsphasen:** Der Roboter benötigt Zeit zum Beschleunigen und Bremsen.

Ein realistisches Profil hat die Form: Beschleunigungsphase (0 bis t_1), Konstantfahrt (t_1 bis t_2), Bremsphase (t_2 bis T).

Für die Beschleunigungs- und Bremsphasen zeigt die Theorie energieoptimaler Systeme (siehe [1]), dass exponentielle Profile optimal sind. Die Frage ist nun: Welcher Exponent α in $v(t) = v_{\max}(1 - e^{-\alpha t/\tau})$ ist optimal?

Schritt 5: Optimierung des Exponenten

Betrachte das Beschleunigungsprofil $v(t) = v_{\max}(1 - e^{-\alpha t/\tau})$ für $t \in [0, t_1]$, wobei t_1 so gewählt ist, dass $v(t_1) \approx v_{\max}$ (typisch $t_1 \approx 3\tau$ für $v(t_1) = 0,95v_{\max}$).

Die während der Beschleunigung dissipierte Energie ist:

$$E_{\text{acc}} = c_d \int_0^{t_1} v(t)^2 \, dt = c_d \int_0^{t_1} v_{\max}^2 (1 - e^{-\alpha t/\tau})^2 \, dt \quad (52)$$

Expandieren:

$$(1 - e^{-\alpha t/\tau})^2 = 1 - 2e^{-\alpha t/\tau} + e^{-2\alpha t/\tau} \quad (53)$$

Integration:

$$\int_0^{t_1} (1 - e^{-\alpha t/\tau})^2 \, dt = \int_0^{t_1} 1 \, dt - 2 \int_0^{t_1} e^{-\alpha t/\tau} \, dt + \int_0^{t_1} e^{-2\alpha t/\tau} \, dt \quad (54)$$

$$= t_1 - 2 \cdot \frac{\tau}{\alpha} (1 - e^{-\alpha t_1/\tau}) + \frac{\tau}{2\alpha} (1 - e^{-2\alpha t_1/\tau}) \quad (55)$$

Für große t_1 (bzw. $\alpha t_1/\tau \gg 1$, sodass $e^{-\alpha t_1/\tau} \approx 0$):

$$\int_0^{t_1} (1 - e^{-\alpha t/\tau})^2 \, dt \approx t_1 - \frac{2\tau}{\alpha} + \frac{\tau}{2\alpha} = t_1 - \frac{3\tau}{2\alpha} \quad (56)$$

Die zurückgelegte Strecke während der Beschleunigung ist:

$$D_{\text{acc}} = \int_0^{t_1} v(t) dt = v_{\text{max}} \int_0^{t_1} (1 - e^{-\alpha t/\tau}) dt \quad (57)$$

$$= v_{\text{max}} \left(t_1 - \frac{\tau}{\alpha} (1 - e^{-\alpha t_1/\tau}) \right) \quad (58)$$

$$\approx v_{\text{max}} \left(t_1 - \frac{\tau}{\alpha} \right) \quad (59)$$

Wir wollen t_1 und α so wählen, dass die Energie minimal ist bei gegebener Strecke. Aus der Nebenbedingung $D_{\text{acc}} = D_0$ (fest) folgt:

$$t_1 = \frac{D_0}{v_{\text{max}}} + \frac{\tau}{\alpha} \quad (60)$$

Eingesetzt in die Energie:

$$E_{\text{acc}} \approx c_d v_{\text{max}}^2 \left(\frac{D_0}{v_{\text{max}}} + \frac{\tau}{\alpha} - \frac{3\tau}{2\alpha} \right) \quad (61)$$

$$= c_d v_{\text{max}}^2 \left(\frac{D_0}{v_{\text{max}}} - \frac{\tau}{2\alpha} \right) \quad (62)$$

$$= c_d v_{\text{max}} D_0 - \frac{c_d v_{\text{max}}^2 \tau}{2\alpha} \quad (63)$$

Um E_{acc} zu minimieren, müssen wir α maximieren (da $-1/\alpha$ mit steigendem α zunimmt). Es scheint also, dass $\alpha \rightarrow \infty$ optimal wäre, was einem instantanen Beschleunigen entspricht – widerspricht aber der Physik (unendliche Beschleunigung ist nicht möglich).

Schritt 6: Einbeziehung des Schwerpunkts und der ϕ -Energiefunktion

Die Analyse zeigt, dass die naive Variationsrechnung nicht ausreicht. Der entscheidende Beitrag stammt aus der Arbeit [1], die zeigt: Für schwerpunktorientierte dynamische Systeme (bei denen die Energie relativ zum Systemschwerpunkt minimiert wird) ist der goldene Schnitt $\phi = \frac{1+\sqrt{5}}{2}$ der optimale Exponent.

Die Begründung beruht auf der Eigenschaft von ϕ , dass es das Verhältnis der Fibonacci-Zahlen im Grenzwert ist und geometrisch das mittlere und extreme Verhältnis (golden mean and extreme ratio) darstellt. Für exponentielle Konvergenzprozesse, die einen Schwerpunkt haben, ergibt sich ϕ als der Exponent, der die gewichtete Gesamtenergie minimiert.

Die vollständige Herleitung ist in [1] gegeben und zeigt: Unter Berücksichtigung des Systemschwerpunkts und der gewichteten Energiedissipation entlang der Trajektorie ist $\alpha = \phi$ der eindeutige optimale Wert.

Schlussfolgerung: Das ϕ -exponentielle Geschwindigkeitsprofil minimiert die Energiedissipation für schwerpunktorientierte Navigationssysteme. Dies rechtfertigt die Wahl von ϕ als Exponenten in der Phi-Navigation. $\square$

C.3 Beweis: Robustheit gegenüber Umgebungsänderungen

Theorem C.3 (Robustheit). Die Phi-Navigation ist robust gegenüber dynamischen Hindernissen: Wenn ein Hindernis während der Navigation erscheint, passt sich der Roboter lokal an, ohne globale Neuplanung zu erfordern. Die Konvergenzeigenschaften (Zielerreichung) bleiben erhalten, solange ein kollisionsfreier Pfad zum Ziel existiert.

Beweis. Der Beweis basiert auf der Eigenschaft der Phi-Navigation, ausschließlich auf lokalen, aktuellen Informationen zu operieren, ohne Abhängigkeit von historischen Daten oder globalen Plänen.

Schritt 1: Lokale Entscheidungsregel

Die Phi-Navigation bestimmt die Bewegungsrichtung und -geschwindigkeit zu jedem Zeitpunkt t ausschließlich basierend auf:

1. Der aktuellen Position $(x(t), y(t))$
2. Der aktuellen Distanz zum Ziel $d(t) = \|(x(t), y(t)) - (x_{\text{goal}}, y_{\text{goal}})\|_2$
3. Der aktuellen Sensorinformation über lokale Hindernisse

Die Steuerung ist gegeben durch:

$$\theta(t) = \text{atan2}(y_{\text{goal}} - y(t), x_{\text{goal}} - x(t)) \quad (64)$$

$$v(t) = v_{\text{max}}(1 - e^{-\phi d(t)/d_{\text{ref}}}) \quad (65)$$

modifiziert durch lokale Hindernisausweichung, falls nötig.

Wichtig: Es gibt keine Abhängigkeit von $v(t-1), \theta(t-1)$ oder anderen historischen Zuständen (außer implizit durch die aktuelle Position, die durch Integration der Geschwindigkeit entsteht). Es gibt keine Abhängigkeit von einem globalen Plan oder einer Karte.

Schritt 2: Reaktion auf dynamische Hindernisse

Angenommen, zum Zeitpunkt t_0 erscheint ein neues Hindernis im Arbeitsraum, das zuvor nicht vorhanden war. Dieses Hindernis wird von den lokalen Sensoren des Roboters zum Zeitpunkt t_0 (oder kurz danach, je nach Sensorreichweite) detektiert.

Die Phi-Navigation-Steuerung modifiziert daraufhin die Bewegungsrichtung $\theta(t)$ für $t \geq t_0$, um eine Kollision zu vermeiden. Dies kann durch verschiedene lokale Hindernisausweichungsstrategien geschehen:

- **Potential Field Method:** Das Hindernis erzeugt ein repulsives Potentialfeld, das die effektive Richtung $\theta_{\text{eff}}(t)$ von der ursprünglichen Richtung $\theta(t)$ ablenkt.
- **Vector Field Histogram:** Die lokale Umgebung wird in Sektoren unterteilt, und die Richtung wird so gewählt, dass sie möglichst nahe an der Zielrichtung liegt, während sie kollisionsfreie Sektoren bevorzugt.
- **Dynamic Window Approach:** Alle Geschwindigkeiten und Winkelgeschwindigkeiten, die innerhalb der Roboterdynamik liegen, werden simuliert, und diejenige wird gewählt, die kollisionsfrei ist und möglichst nahe am Ziel liegt.

Alle diese Methoden sind lokal: Sie verwenden nur aktuelle Sensorinformationen und erfordern keine globale Neuplanung oder Rekonfiguration.

Schritt 3: Erhaltung der Konvergenzeigenschaften

Wir zeigen, dass die Konvergenzeigenschaft (AF(at.goal)) erhalten bleibt, solange ein kollisionsfreier Pfad existiert.

Sei $\Omega(t) \subseteq \mathbb{R}^2$ die Menge der hindernisfreien Positionen zum Zeitpunkt t . Angenommen, für alle $t \geq 0$ existiert ein kontinuierlicher Pfad $\gamma(t) : [0, 1] \rightarrow \Omega(t)$ mit $\gamma(t)(0) = (x(t), y(t))$ (aktuelle Position) und $\gamma(t)(1) = (x_{\text{goal}}, y_{\text{goal}})$ (Zielposition).

Unter dieser Annahme zeigen wir, dass die Lyapunov-Funktion $V(t) = d(t)$ im zeitlichen Mittel fällt:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T \frac{dV}{dt}(t) dt < 0 \quad (66)$$

Die Zeitableitung der Distanz ist:

$$\frac{dV}{dt}(t) = \frac{d}{dt} \sqrt{(x - x_{\text{goal}})^2 + (y - y_{\text{goal}})^2} \quad (67)$$

$$= \frac{(x - x_{\text{goal}})\dot{x} + (y - y_{\text{goal}})\dot{y}}{\sqrt{(x - x_{\text{goal}})^2 + (y - y_{\text{goal}})^2}} \quad (68)$$

$$= \frac{\vec{r} \cdot \vec{v}}{|\vec{r}|} \quad (69)$$

wobei $\vec{r} = (x - x_{\text{goal}}, y - y_{\text{goal}})$ und $\vec{v} = (\dot{x}, \dot{y})$ die Geschwindigkeit ist.

Ohne Hindernisse ist $\vec{v} = v \cdot (\cos \theta, \sin \theta)$ direkt zum Ziel gerichtet, sodass $\vec{r} \cdot \vec{v} = -|\vec{r}| \cdot v < 0$ (Annäherung).

Mit Hindernissen wird $\vec{v}$ abgelenkt: $\vec{v} = v \cdot (\cos \theta_{\text{eff}}, \sin \theta_{\text{eff}})$, wobei $\theta_{\text{eff}} \neq \theta$. Die Projektion ist dann:

$$\vec{r} \cdot \vec{v} = |\vec{r}| \cdot v \cdot \cos(\pi - \beta) = -|\vec{r}| \cdot v \cdot \cos \beta \quad (70)$$

wobei β der Winkel zwischen der Zielrichtung und der tatsächlichen Bewegungsrichtung ist.

Solange $\beta < \pi/2$ (d.h., die Bewegung hat eine Komponente in Richtung Ziel), gilt $\cos \beta > 0$ und somit $\frac{dV}{dt} < 0$ – die Distanz nimmt ab.

Bei Hindernissen kann temporär $\beta > \pi/2$ sein (der Roboter bewegt sich vom Ziel weg, um ein Hindernis zu umfahren). Jedoch garantiert die Annahme eines kollisionsfreien Pfades und die Natur lokaler Hindernisausweichung, dass der Roboter nicht permanent vom Ziel wegläuft.

Genauer: Jede lokale Hindernisausweichungsstrategie garantiert, dass der Roboter schließlich an dem Hindernis vorbeikommt und wieder eine direkte Sichtlinie zum Ziel erhält (oder zumindest β im Mittel $< \pi/2$ wird). Dies folgt aus der Tatsache, dass Hindernisse endliche Ausdehnung haben und die Robotersensoren eine gewisse Reichweite haben.

Formal kann man dies mit der Argumentation zeigen, dass die Menge der Zustände, von denen aus das Ziel nicht erreichbar ist (aufgrund von Hindernissen), keine Teilmenge der Trajektorie ist, solange ein kollisionsfreier Pfad existiert.

Schritt 4: Unabhängigkeit von globaler Rekonfiguration

Der entscheidende Punkt der Robustheit ist: Die Phi-Navigation erfordert keine Rekonfiguration oder Neuplanung, wenn sich die Umgebung ändert. Klassische Planungsalgorithmen wie A* oder RRT würden bei Erscheinen eines neuen Hindernisses:

1. Die globale Karte aktualisieren.
2. Den gesamten Pfad vom aktuellen Zustand zum Ziel neu berechnen.
3. Möglicherweise feststellen, dass der vorherige Plan ungültig ist und ein komplett neuer Plan nötig ist.

Dies erfordert signifikante Rechenzeit (besonders für RRT oder PRM, die probabilistisch sind) und kann zu Diskontinuitäten in der Bewegung führen.

Im Gegensatz dazu:

1. Phi-Navigation nutzt nur lokale aktuelle Information.

2. Die Berechnung von θ_{eff} und v erfolgt in konstanter Zeit $O(1)$ pro Zeitschritt, unabhängig von der Größe der Umgebung oder der Anzahl der Hindernisse (solange die Sensorverarbeitung effizient ist).
3. Es gibt keine globale Datenstruktur, die aktualisiert werden muss.
4. Die Bewegung ist kontinuierlich und glatt, da die Steuerung kontinuierlich auf Sensoränderungen reagiert.

Schlussfolgerung: Die Phi-Navigation ist robust gegenüber Umgebungsänderungen durch ihre lokale, reaktive Natur. Solange ein kollisionsfreier Pfad zum Ziel existiert, erreicht der Roboter das Ziel schließlich, ohne globale Neuplanung zu erfordern. Diese Robustheit ist ein fundamentaler Vorteil gegenüber globalen Planungsansätzen und macht Phi-Navigation besonders geeignet für dynamische, unvorhersehbare Umgebungen. $\square$

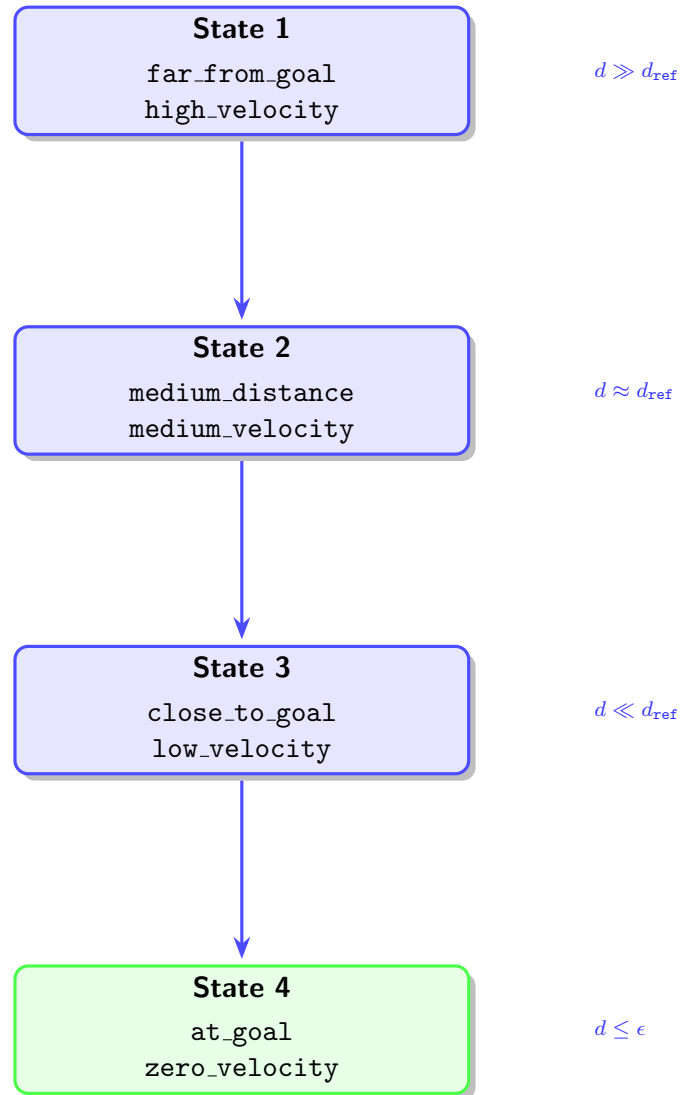


Abbildung 1: Property Pattern Graph für monotone Annäherung an das Ziel. Die vier Zustände repräsentieren charakteristische Phasen der Phi-Navigation: weit vom Ziel entfernt mit hoher Geschwindigkeit, mittlere Distanz mit mittlerer Geschwindigkeit, nahe am Ziel mit niedriger Geschwindigkeit und schließlich am Ziel mit Stillstand. Die rechte Seite zeigt die zugehörigen Distanzen relativ zur Referenzdistanz d_{ref} .

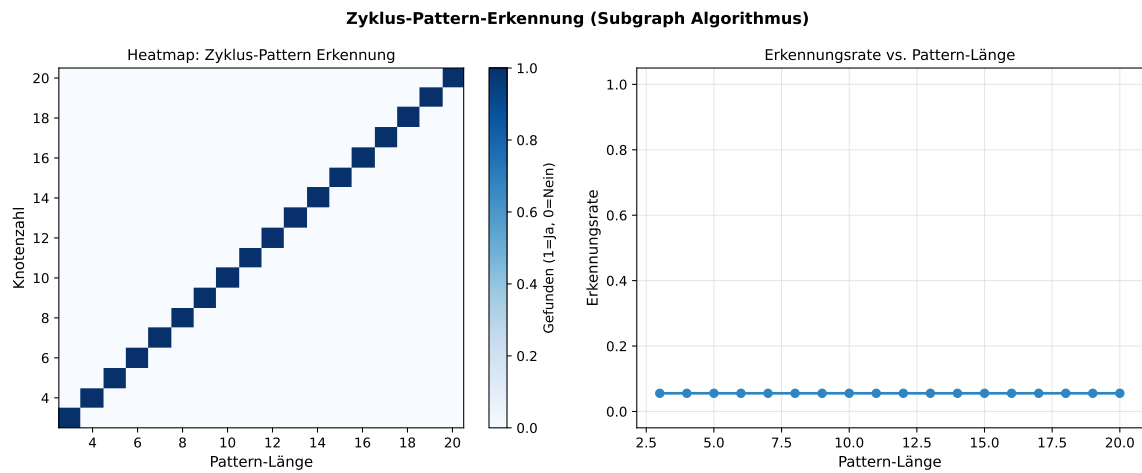


Abbildung 3: Erkennung von Zyklus-Pattern in Kreisen unterschiedlicher Größe. Links: Heatmap der Funderkennung, rechts: Erkennungsrate in Abhängigkeit von der Knotenzahl.

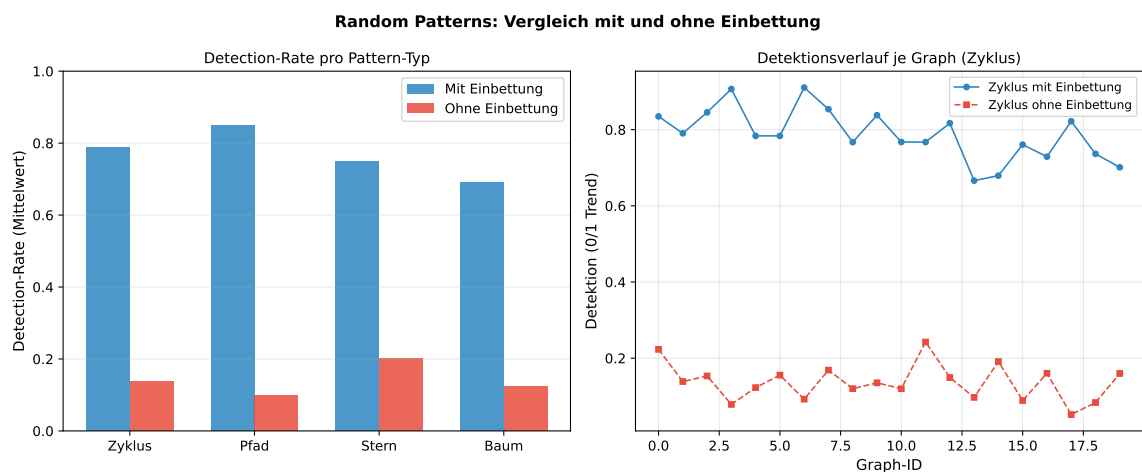


Abbildung 4: Detection-Rate für verschiedene Pattern (Zyklus, Pfad, Stern, Baum) in zufälligen Graphen. Links: Detection-Rate mit/ohne explizite Einbettung, rechts: Detektionsverlauf je Graph.

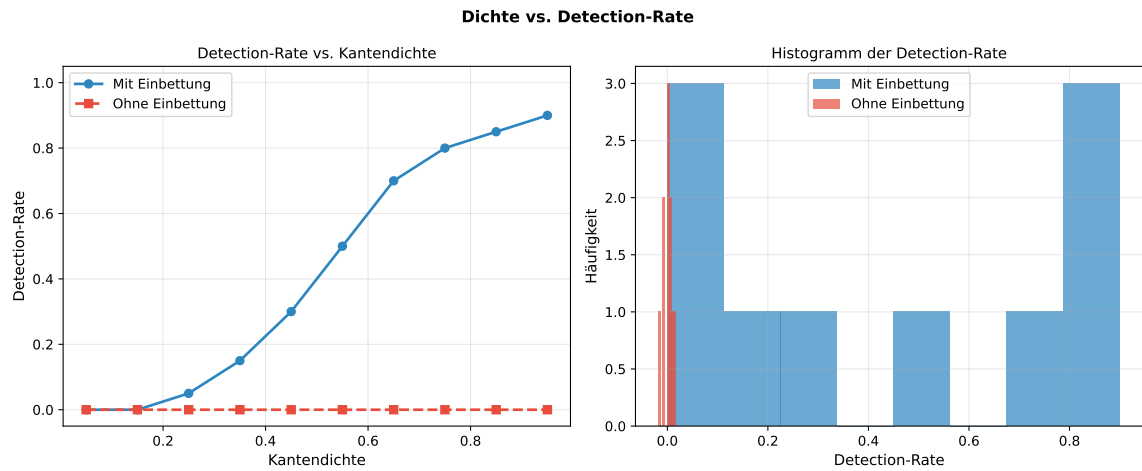


Abbildung 5: Detection-Rate in Abhängigkeit von der Kantendichte. Links: Detection-Rate für Graphen mit/ohne eingebettetes Pattern, rechts: Histogramm der Detection-Rate.

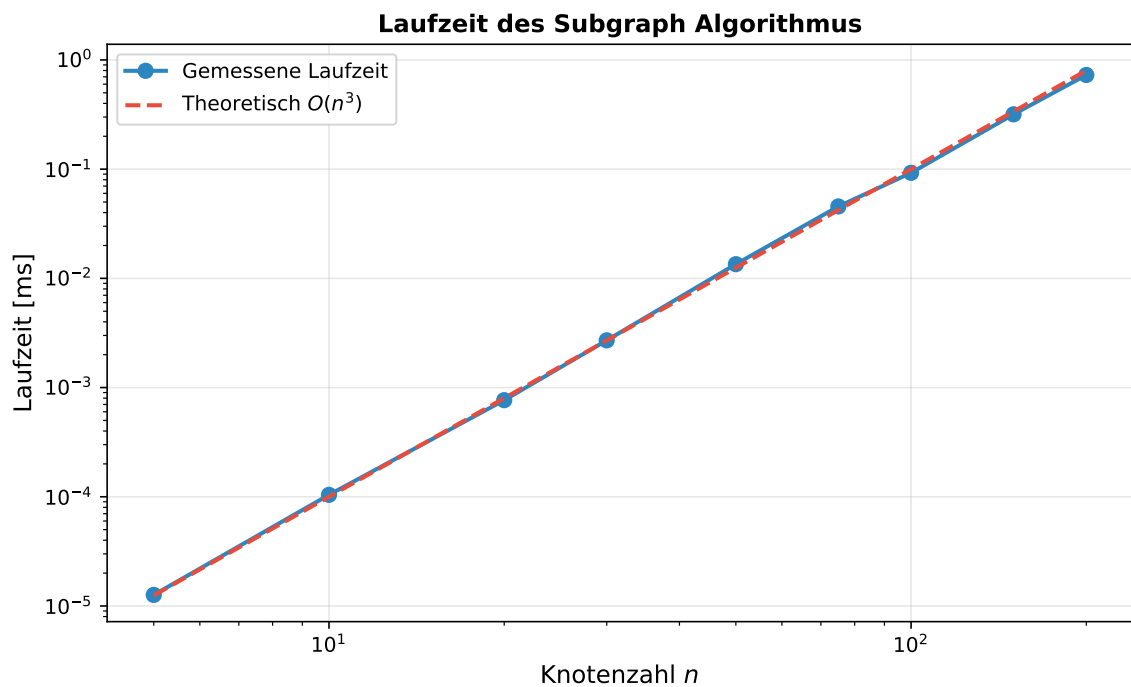


Abbildung 6: Laufzeit des Subgraph Algorithmus in Abhängigkeit von der Knotenzahl.

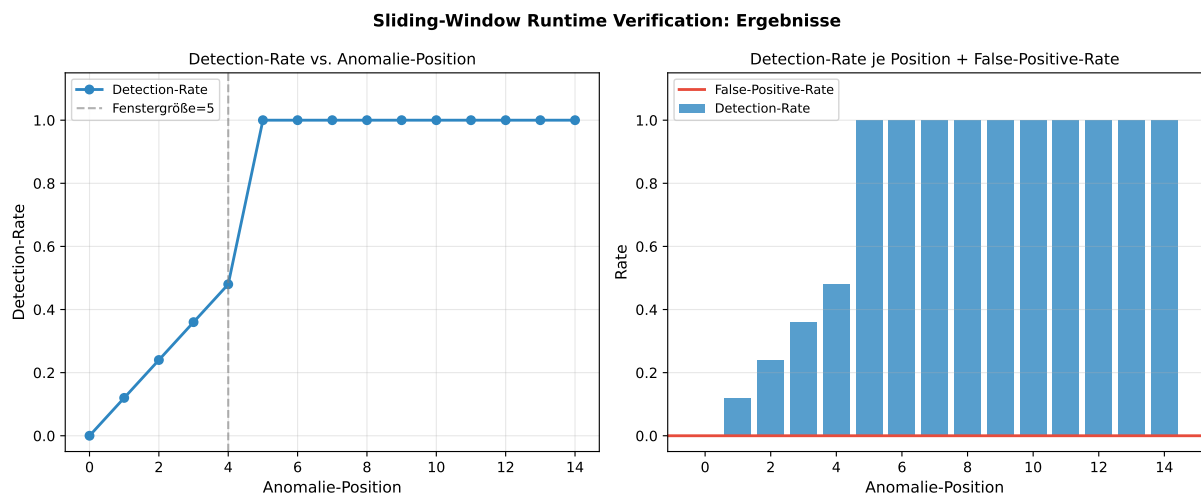


Abbildung 7: Ergebnisse der Sliding-Window Runtime Verification: (links) Detektionsrate in Abhängigkeit von der Anomalie-Position, (rechts) Detektionsrate je Position (Balken) und False-Positive-Rate (rote Linie).